

21 FOR 21

Twenty-One Days · Twenty-One Million Sats · Zero Human Lines

The complete post-mortem of the Bitcoin Machine Code experiment

A full validating Bitcoin node written entirely in AI-authored x86-64 assembly
first vectors 2026-08-11 · 21 days of git history to 2026-09-02

Contents

PART I: THE FIRST WEEK – LEARNING x86-64 AT THE SPEED BITCOIN TEACHES	..7
Chapter 1. SHA-256, and why the oracle is the only witness you trust.....	7
Chapter 2. The golden rules – each one a scar.....	8
Chapter 3. RIPEMD-160, or: read the reference implementation.....	10
PART II: THE CONSENSUS MUSEUM – defects in the script machine	11
Chapter 4. First contact with the real chain.....	11
Chapter 5. The false-accept horizon.....	13
Chapter 6. The interpreter as archaeology.....	16
PART III: THE LEDGER ENGINE – storage as a contact sport	18
Chapter 7. The store that wrote the disk thirteen times.....	18
Chapter 8. Crash-consistency, three times over.....	19
Chapter 9. When the counter lies, and when the set does.....	21
Chapter 10. The capstone.....	22
PART IV: THE HUNT FOR CYCLES – performance in the land of Amdahl	24
Chapter 11. The profile that started the war.....	24
Chapter 12. $O(n^3)$: the biggest win was a data structure nobody had deleted.....	25
Chapter 13. The multiply at the bottom of the world.....	26
Chapter 14. The bottleneck is not arithmetic, it is serialization.....	28
Chapter 15. The GPU that was never needed.....	29
PART V: THE NETWORK STRIKES BACK – P2P, keep-up, and three bad days	...30
Chapter 16. "It only advances when restarted" – incident #33.....	30
Chapter 17. The units audit, and the four cap bugs.....	31
Chapter 18. connect=, or the config option that lied.....	31
Chapter 19. The serve path: two masks, three complicit tests.....	32
Chapter 20. Relay, the orphan pool, and the mempool freeze at 4096.....	33
Chapter 21. The privacy, anonymity, and misbehaviour arc.....	33
PART VI: THE LEDGER LIES ONCE, TWICE – the 2026-09-01 double incident	.35
Chapter 22. The genesis-first answer.....	35
Chapter 23. The 2,596 resurrected coins.....	36
Chapter 24. The awk command that froze the host.....	38
PART VII: DEALING WITH YOUR HUMAN – the operator as an API, and as a mirror	40
Chapter 25. The contract: three modes of authorization.....	40
Chapter 26. The human's three contributions.....	41
Chapter 27. What the machine learned about its human.....	43
Chapter 28. The machine is a chorus.....	43
Chapter 29. The mirror test.....	44
PART VIII: THE AUDITORS – what independent review found, and what fixing it broke	46

Chapter 30. The audit, and the honesty of its framing.....	46
Chapter 31. The response, and the three remediation incidents.....	46
Chapter 32. What the auditors verified, and what they declined.....	48
PART IX: THE LEDGER VS THE GIANT – what is missing, what is better, and the honest numbers	49
Chapter 33. What is honestly missing, and why it wasn't built.....	49
Chapter 34. Is anything better than Bitcoin Core?.....	50
Chapter 35. Why write this report now.....	52
PART X: CAN A HUMAN DO THIS? – and what intelligence is for	54
Chapter 36. The arithmetic of twenty-one days.....	54
Chapter 37. What actually produced the speed.....	55
Chapter 38. Three philosophical footnotes, written by a project that could not help it.....	57
CHAPTER: THE MODELS – who actually typed this	56
PART XI: WHAT WE'D DO DIFFERENTLY – process regrets, not bug regrets ..	59
Appendix A: THE NUMBERS, ON ONE PAGE	60
Appendix B: HOW TO READ A LOG THAT WAS WRITTEN AS A VACCINE	61
Appendix C: READING ORDER – AUDITING THIS REPORT AGAINST THE SOURCE ...	62
Epilogue: THE NODE IS STILL RUNNING	63
Appendix D: WHAT WE LEARNED, IN ONE PAGE	64

Prologue: The Premise

The experiment is stated in one line at the top of the project's plan file:

```
"Goal: a working Bitcoin client for Linux implemented as x86-64 assembly, every  
line authored by an AI (no human code). The security-critical crypto (SHA-256,  
secp256k1, ECDSA) lives in raw assembly."
```

Not a toy client. Not a signing library. A full-validating Bitcoin node: headers-first initial block download, script verification of every signature in every historical block, a chain-scale UTXO set, a mempool with relay policy, an inbound serving path, JSON-RPC, a wallet. And not in C with a little assembly – the assembly is the software. The C that exists is, in the project's own words, "orchestration" and "proving scaffolding": harnesses and glue whose job is to demonstrate that the machine code agrees with trusted references.

Twenty-one days. That is the full span, and the reader should hold the number next to the other number this report is named for: twenty-one million. Twenty-one days from the first SHA-256 test vector to a node holding every one of the 165,726,554 property records that stand between a holder and their share of twenty-one million bitcoins – verified, hashed, and proven equal to Bitcoin Core's answer, to the satoshi, at a chain height of 963,967. The coincidence of numbers is not lost on the authors. The project's own README records the shape of it: a single daemon, "about 290 test binaries" behind every merge, verifying the entire chain. This report adds the arithmetic the project left implicit – the title's own joke: twenty-one days of work against twenty-one million sats at stake, zero human-written lines. It is a pairing, not an equivalence, and this report earns its halves separately: 21 days is the git span, the conservative clock, with the two pre-history days named; 21 million is the monetary supply every block is quietly rationing against. Early drafts of this cover carried an instruction count – "one million," then the measured 263,437 – and each time the number was an argument this report had to win before its first page. The title wins by not making one: a pairing, not a boast, and every number that matters is printed with its method inside, not on the cover.

Whether that number is an achievement, an anomaly, or an early warning is Part X of this report. The evidence for that judgement is assembled by everything that comes before it.

Three properties of Bitcoin make this a strange and a beautiful target at the same time.

Bitcoin is fully specified by its history. There is no ambiguity to argue about: the chain is a million-block answer key. Every block ever mined is a test vector with a verdict already stamped on it, and the network running Bitcoin Core – the reference implementation – will tell you, byte by byte, what the right answer to any input is. A node written from scratch has a free, omniscient, permanently-available examiner sitting in the next directory over. (For most of the project's life that examiner was a scratch build at /storage/core-oracle, with the production Core install explicitly off-limits – "Running

Core for benchmarking and as an oracle is authorized as of 2026-08-21 – from the separate source build, never the production install.")

Bitcoin's rules are a museum of one-bit decisions. The chain's script language carries decades of archaeology: disabled opcodes, soft forks that narrowed old rules, flags that turn on at specific heights, two blocks on mainnet that violate their own activation rules and are exempted by *hash*, a sighash algorithm with a famously broken SIGHASH_SINGLE case that the consensus preserves forever because fixing it would be a fork. Any implementation that is not Bitcoin Core must absorb every one of those quirks exactly, or it is not a Bitcoin node; it is a machine for disagreeing with money.

And assembly leaves nowhere to hide. Every bookkeeping rule that a C compiler enforces for you – stack alignment, callee-saved registers, struct layout, evaluation order – becomes the author's responsibility, per function, per instruction. When an AI writes that assembly, you find out precisely which things an AI is confident about and which things it is confidently wrong about.

The project ran for three calendar weeks of wall-clock time – from the first SHA-256 vectors on 2026-08-11 to the last commits on 2026-09-02, the day this report was compiled. In that time it: built an ECDSA verifier from field arithmetic up; wrote a complete script interpreter; synced the real ~963,000 block mainnet archive; replayed the entire chain verifying every signature and proving, at the end, that its UTXO set was byte-identical to Bitcoin Core's down to the MuHash3072 digest; served real peers; grew 155 JSON-RPC methods, a mempool, three indexes, a wallet, Tor/I2P/CJDNS transports and BIP324 encrypted transport; and ran a live systemd service on mainnet following the tip continuously.

It also produced, by its own count, more than fifty numbered production incidents – defects whose first existence was in a running system – including at least nine consensus false-accepts in the chain-splitting direction, a hard-frozen host, an archive that had been silently storing 482,000 blocks without their witness data, and 2,596 already-spent coins resurrected into the chainstate of a live node.

This report is the story of both columns of that ledger, told mostly in the project's own words, because the project kept unusually honest records. Its own README states the ground rule: "A clean replay proves only that no real block was refused." Everything that mattered was learned by trying to be wrong in public and measuring it.

A final framing quote, from the project's honest-assessment document, written three days in with the replay at height ~721,000:

"This is a consensus-verification engine, not a node. It cannot replace Bitcoin Core for any real user today. ... Its consensus correctness is being actively established and is not yet established: forty-two defects have been found ... and the discovery rate is not yet decelerating. The most recent five were found *after* a clean genesis-to-963,000 replay, by differential testing against Core rather

than by replaying – which is the point: a clean replay proves only that no real block was refused."

That paragraph is the thesis of the whole experiment. The rest of this report is the evidence for it.

PART I: THE FIRST WEEK – LEARNING x86-64 AT THE SPEED BITCOIN TEACHES

Chapter 1. SHA-256, and why the oracle is the only witness you trust

The build order was dictated by dependency, not ambition: SHA-256 first ("sha256.asm: full SHA-256 (init, one-block compression, one-shot with padding). VERIFIED 7/7 PASS"), then secp256k1 field arithmetic, then point arithmetic, then scalars, then ECDSA verify. Each primitive was proven against a Python big-int or ctypes oracle before the next layer was allowed to exist. This was not ceremony. It was the only mechanism that repeatedly caught what nothing else could.

The first lesson that earned permanent status cost days and came from a hand-typed constant.

fe_inv – the field inverse, computed as $a^{(p-2)}$ by square-and-multiply – returned wrong values, and the failure was maddening because the two primitives it was built from, fe_mul and fe_sqr, were individually proven. The log's debug trail is a model of bisection: a pure-C loop driving the same assembly produced the *same* wrong answer (so not the asm loop control); 300 chained in-place squarings matched Python exactly (so not fe_sqr); a checkpointed trace diverged "at i=16 onward" – and then the resolution:

"NOT a fe_mul/fe_sqr bug at all. The crypto primitives were always correct. The bug was a hand-typed EXPONENT TABLE: in BOTH my C test and the asm EXP_BYTES I wrote the p-2 little-endian bytes with the 0xFE byte at offset 3 instead of offset 4."

The verifier's test and the implementation had been *both* written from the same faulty transcription, and so agreed with each other and disagreed with mathematics. The rule that came out of it is the first of the project's "golden rules":

"NEVER hand-type multi-byte constant tables; derive/verify them from Python (the oracle) first, and prefer a Python-driven ctypes test over hand-embedded constants for cross-checking."

This rule will reappear throughout this report under one general form, which is this report's recurring villain: *a reference derived from the implementation instead of from the specification*. Three weeks later, an entire class of consensus bugs was still being explained by it.

The same week, point_add failed silently for hours. The cause was the language itself:

```
"SUBTLE NASM GOTCHA: ';' is a COMMENT in NASM, NOT a statement separator. ... I wrote the equal-X comparison block as one-liners such as mov rax,[rbp-0x90+0]; cmp rax,[rbp-0xb0+0]; jne .distinct In NASM only the FIRST instruction (mov) assembled; everything after the first ';' became a COMMENT, silently dropping the cmp/jne."
```

An AI author with C and Python priors, writing what it believes is a multi-statement line, is instead commenting out its own control flow. The assembler accepts this in complete silence. Golden rule #2: one instruction per line, always.

Chapter 2. The golden rules – each one a scar

By mid-August the LOG.md "KEY FACTS" block reads like a war memorial, and every entry maps to a specific crash. They are worth collecting in one place because they are the first deliverable of the whole experiment: a precise list of the things an AI (or anyone) gets wrong in x86-64, ranked by how much pain each one caused.

THE SAVE-AREA RULE. The single most expensive rule. On entry, the code pushes callee-saved registers into [rbp-8..rbp-40]; scratch locals placed in that window silently corrupt the *caller's* saved registers, and the crash appears somewhere else entirely, later:

```
"GOLDEN RULE AGAIN: store_append's 48-byte index-record buffer sat at rbp-0x50, whose top (rbp-0x21) CROSSED the callee-saved save area. The record write silently clobbered saved r12-r15/rbx; when store_append returned, main's callee-saved registers were garbage -> later crash in store_get_tip."
```

The project's discipline after being bitten repeatedly: "before writing ANY asm fn, list every callee-saved reg touched, ensure each is pushed, and place every local strictly below the save area." It still recurred. Decades-old assembly wisdom, violated weekly.

THE FIELD-WIDTH RULE. A 64-bit load of a 32-bit field reads its neighbor:

```
"when a dword field sits next to a qword field in a frame, ALWAYS size the load to the field" – from a bug where mov rax,[rbp-0x34] pulling an 8-byte window over a 4-byte n field produced a txid-slot index of (idx<<32)|n, "garbage, desyncing the walk and writing far out of bounds." This same bug returned months later as an interpreter defect: OP_SIZE read a uint32 length field with mov rdx,[r13], pulled four bytes of stack into the operand, and the existing test missed it because "its specific data happened to have all-zero padding that hid the corruption."
```

THE POINTER-IN-REGISTER RULE (the r13 leak). The project's own deep hash chain (block_hash -> sha256d -> sha256_full) had, at one point, a callee that leaked r13. Once discovered it became a load-bearing convention – keep the loop's pointer in a stack slot and reload it, never in r13 across a call:

"node_ibd_blocks FIRST version kept the header-store pointer in r13 across the per-header loop. The deep block_hash chain LEAKS r13 ... by i=1 hst_get_at was called with a garbage hst pointer." Diagnosed with "a single-char stderr tracer that showed the exact cut."

THE BYTE-ORDER RULES. Ports are big-endian in the sockaddr, "passing a raw 8333 made sin_port = 0x8D20 (=36128) -> ECONNREFUSED". Block hashes exist in two orders (raw digest vs display) and the genesis block is famous for being printed both ways; the hash->height index "keyed the index with the WRONG byte order: index.dat stores hashes in BE display order but the getdata/inv wire hash is LE. getdata therefore missed every hash." RIPEMD-160 uses little-endian message words where SHA-256 uses big-endian – and "I had bswap'd, copying the SHA habit."

THE NUL RULE. A filename built at runtime from pieces must be terminated explicitly; a dword store of ".dat" writes exactly four bytes:

"open(\"blk00000.dat\\375\\177\"...) – stack garbage read past the end... Writing a filename suffix with a DWORD store covers only 4 bytes -- ALWAYS write the terminating NUL explicitly when building a runtime filename, or open() will read past the end into stack garbage and create corrupted long filenames."

THE POLL() RULE, learned on the live network: "poll() returns as soon as the FIRST descriptor is ready – not after the timeout. So one nearby peer answering in ~20 ms made the call return immediately, and every other candidate was judged un-ready in the loop below and closed on the spot. The 2500 ms budget was never actually spent." One node kept exactly one peer of eighty-five because of this. The comment above the broken code, the log notes, "describes the intent exactly, and the intent is what does not survive poll()'s return semantics."

THE STRUCT-OFFSET RULE, from a night where three ports of C code to assembly all failed identically:

"1. tapctx_t field ORDER. The first offsetof probe I wrote put hash_type after num_inputs; the real struct has it immediately after n_in. 2. txview_t layout. I assumed it matched bitcoin_segwit.c's swtx_t. It does not... The twin segfaulted on its first sha256_outputs. 3. The SysV register/stack boundary. ts_agg_hashes takes eight arguments; the sixth (h_seq) goes in r9, and I read it from [rbp+16], which is the SEVENTH."

"Standing rule after tonight, now applied without exception: no struct offset, no field order, and no argument position goes into assembly until it has been printed by an offsetof probe or counted against the SysV tables. Three for three tonight."

The diagnostic pattern in case 3 became a reusable law: "ASAN reported NO memory error while the differential reported 17 identical failures. A segfault plus clean sanitizer plus a systematic wrong-value pattern is a marshaling bug, not a memory bug – and the argument list is the first place to count."

Chapter 3. RIPEMD-160, or: read the reference implementation

RIPEMD-160 deserves its own chapter because it is the purest example of the failure mode this experiment existed to study. The digest was deterministically wrong – and a Python transcription written independently agreed with the wrong asm. Two implementations, same author, same mistake, perfect confidence:

"the way out was fetching the authoritative pycryptodome src/RIPEMD160.c and reading the FINAL MIXING stage literally: the left/right line terms are crossed in the reference ($h1 + CL + DR$, not $h1 + cc + d$ as I had it)."

Nothing in the RIPEMD-160 specification-as-memory tells you that the final compose crosses the lines. Nothing in a test corpus of your own making tells you either, because the corpus inherits the same crossed belief. The only events that ended the debugging were *external ground truth*: someone else's correct code, read literally.

The general form – which the project eventually wrote into its own documentation as the method of record – is that agreement between your implementation and your test suite is not evidence. Ground truth must come from outside the belief boundary: a reference implementation you did not write, published vectors you did not generate, or Bitcoin Core itself.

PART II: THE CONSENSUS MUSEUM – defects in the script machine

Chapter 4. First contact with the real chain

By August 17 the project had the shape of a node: assembly sockets, assembly P2P framing, assembly tx parsing, assembly consensus checks (PoW, merkle root), a durable store, and a download orchestrator pulling the real ~962,000 block mainnet archive from eight-plus distinct internet peers simultaneously ("the per-worker download LOOP IN ASSEMBLY"). What it did not yet have was any script verification beyond the ECDSA primitive, and no UTXO set. The design phase for those came to be called Stage D: replay the whole archive, verifying every signature in every historical block, and build the UTXO set as you go – the hardest certification a Bitcoin implementation can attempt, because it walks through every rule change since 2009 in order, in one pass, with no assumevalid shortcut.

Stage D, when it was wired live on 2026-08-19/20, produced three real production incidents in its first hours. The numbering restarted – these were the project's first *consensus* incidents, the kind that only exist once a node is pointed at the real chain.

INCIDENT #1: THE LSM INVERTED ITS OWN MANIFEST. The UTXO store had just been rebuilt as an LSM-tree (the story of why is Part III of this report). At real mainnet height 184390 the replay rejected a block – "legacy script verification failed" – and the daemon then did something that the log later learned to be suspicious about generally:

```
"immediately followed by a blind auto-triggered 'in-place recovery' compaction
(daemon/main.c calls utxo_live_recover() unconditionally on ANY utxo_live_catchup
failure -- it cannot distinguish 'manifest full' from 'genuine consensus
rejection', a still-present design pattern worth treating any future
FATAL+recovery pair with suspicion over)."
```

The root cause, found by reading bitcoin_utxo_lsm.asm directly: the run scan walks the manifest from highest index (newest) down, and compaction appended its merged *oldest* run at the *highest* index. "So a stale, already-deleted key could resolve as live again post-compaction." The regression test was verified the project's now-standard way: "verified via git stash that it FAILS against the pre-fix code with the exact predicted symptom (the deleted key resolving live again) and PASSES with the fix."

INCIDENT #2: DANGLING POINTER INTO A GROWABLE POOL. The same rejection survived the fix – proof, recorded gratefully, that incident #1 "while real, was not the (sole) cause." The reproduction technique became canonical: real archive block files symlinked into a throwaway directory, the catch-up run against fresh state there, "never touching the live daemon's own data," plus debug

instrumentation proving the resolved prevout script was "CORRECT at resolve time and CORRUPTED at verify time for the identical input." The cause: a byte-pool bump allocator whose backing realloc() relocated the buffer while earlier inputs still held raw pointers into it – and because whether realloc moves a buffer depends on heap layout, "it reproduced inconsistently between attempts," the signature of the class. The fix – store a stable byte OFFSET, resolve to a pointer only after the pool stops growing – became one of the project's named engineering rules: "A pointer into a growable buffer dies at the next growth."

INCIDENT #3: THE OVERNIGHT OPCODE MYSTERY. With standing authorization to fix and redeploy autonomously while the human slept, the daemon hit a new deterministic rejection at height 251683. The failing script decoded to a "hash puzzle" scriptPubKey – OP_SIZE OP_DUP OP_1 OP_GREATERTHAN OP_VERIFY OP_NEGATE OP_HASH256 OP_HASH160 OP_SHA1 OP_SHA256 OP_RIPEMD160 OP_EQUAL – a legitimately-spendable script nobody had ever tried to verify before. Two independent interpreter bugs:

"1. OP_SIZE read the top stack element's length via mov rdx,[r13] -- a 64-bit load -- but that field is a uint32 ... pulling in 4 bytes of DATA as garbage high bits ... The existing OP_SIZE test vector never caught it because it never decoded the pushed size back as a number, and its specific data happened to have all-zero padding that hid the corruption." "2. OP_SHA1 was entirely unimplemented ('; SHA1 not available -> bad opcode') -- a real, always-defined Script opcode, never built. No SHA-1 existed anywhere in this codebase."

A SHA-1 implementation had to be written from scratch – in assembly, that night – because an opcode the interpreter had cheerfully declared "bad" was in use on mainnet since 2013. The lesson generalizes beyond opcodes: an interpreter is not correct because its *tested* opcodes work; it is correct only when someone has enumerated every entry in the table and asked whether the untested half exists.

INCIDENTS #4 AND #5 (08-21), the overnight replay's next two teachers. At height 388431 the replay rejected a real block because "OP_CLTV/OP_CSV in bitcoin_interp.asm were wired as pass-through no-ops rather than the real BIP65/BIP112 locktime/sequence comparisons – silently accepting scripts a real node must reject." Timelocks, two of the most load-bearing consensus rules in every Lightning and HTLC deployment, were decorative. The redeploy decision matters as an artifact of engineering character: rather than trust the in-place recovery path, the operator dropped the entire UTXO state and replayed from zero, "the safer call given the project's standing 'verify against real data, don't assume' discipline."

Incident #5 was the crash-resume lesson in its purest form. An unrelated host reboot killed the daemon mid-replay; the restart re-verified a block whose inputs it had already durably spent, and reported it as a consensus rejection:

"applied_height was only persisted at rare compaction events or once at the very end of a (possibly hours-long) catch-up call, while every individual utxo_lsm_put/

del is already durable the instant it runs. An unclean process death therefore routinely left the true on-disk UTXO state hours AHEAD of the last-written checkpoint."

The fix is a sentence worth framing in any storage course: persist the applied height "after every successfully applied block, not just at compactions / end-of-call, so no gap between real and checkpointed state can ever exist – correct by construction rather than by detecting and tolerating the gap after the fact." The regression test forks a child that applies exactly one block and `_exit()`s "right where its checkpoint should persist," restarts, and asserts a clean no-op – and the negative control confirms the test reproduces "the exact production symptom" when the fix is disabled.

Chapter 5. The false-accept horizon

If Part I taught lessons about assembly, Part II's core lesson is about epistemology, and it arrives with numbers attached.

The project's own assessment document states it flatly:

"Most were false rejects – the safe direction. They stop the replay loudly and cost time, not correctness. At least five were false accepts – the chain-splitting direction: soft forks activating one block late (#6), a 521-byte tapscript stack item (#19), two BIP341 sighash shapes (#23, #24), and an unenforced BIP66 signature encoding rule ... Several were reachable only by asking Core, never by replaying the chain. No such transaction exists in the chain, because Core-running miners never mined one. The replay could have run to tip, clean, with all of them still present."

That is the sentence every auditor, every reviewer, and every person who ever said "the tests pass" should read once slowly. A genesis-to-tip replay of real mainnet data – the single most impressive self-test this project ran, and the one that certifies the UTXO set byte-for-byte against Core – is *blind* to the most dangerous class of defect, because the dangerous inputs are precisely the ones the honest network never mined.

THE CANONICAL FALSE ACCEPT: INCIDENT #6, THE GENESIS THAT WASN'T. Discovered while wiring RPCs, not by the replay at all: "index.dat record 0 was 00000000839a8e... -- block 1." The genesis block had never been stored. And because soft-fork activation is height-computed from the archive's own heights, everything was shifted by one:

"DERSIG (363725), CLTV (388381), CSV (419328) and NULLDUMMY (481824) each MISSING their flag bit at their own activation block. For exactly one block per boundary we applied looser rules than Core: a chain-split in the accept direction."

The deeper part is why the replay could not see it: "looser rules accept a superset, and real chain data is valid under the strict rules anyway – which is a genuine limit on what a clean replay proves." And the recovery story is pure Bitcoin folklore: the user proposed re-downloading the chain, which "could

never have worked: the P2P 'from the beginning' locator is the all-zero hash and peers answer from block 1; genesis is never transmitted." The most famous block in financial history is the only one that cannot be downloaded from the network. It had to be injected from its own constant, 285 bytes appended, the index shifted by one record, chainwork rebuilt.

THE SPECTACULAR ONE: INCIDENT #10, THE WITNESS-STRIPPED ARCHIVE. On 08-22, at 06:13, the from-scratch replay rejected the first segwit block, 481824: "p2wpkh needs exactly 2 witness items. The Core oracle shows that input has exactly 2." The bytes being verified differed from the bytes Core had – because the node had been requesting inventory type MSG_BLOCK (2) instead of MSG_WITNESS_BLOCK (BIP144), and per BIP144 a peer answers that with "the non-witness serialization."

"The merkle root commits to txids, which are computed over that same stripped form, so cons_verify accepted every stripped block, 'integrity OK' was true of headers and hashes and hollow for bodies, and cuda_txid_reindex's 'merkle OK' never could have caught it. Core *would* have rejected each of those blocks on arrival -- bad-witness-nonce-size – because Core validates the BIP141 witness commitment. This node did not. Two bugs, then: the request type, and a missing consensus check."

Every block from segwit activation to tip – roughly 482,000 blocks, some 480 gigabytes – had been stored as a shadow of itself, and every tool that "verified the archive against Core" had compared exactly the fields that stripping does not change. The recovery is Part III's material (truncate to block 481,823, re-fetch the tail at ~14 MB/s from a local Core oracle bound to sixteen loopback aliases so sixteen parallel workers could each have a peer). The epitaph is the lesson:

"The lesson is the one from #6 again, sharper: a replay that runs clean is evidence only about the checks that exist."

THE BUGS THAT ONLY CORE COULD SEE. Incidents #23/#24, the BIP341 sighash pair: hash_type was never validated (a signature's last byte, entirely "attacker-chosen," and hash_type 0x04 produced a valid-looking sighash where Core errors out), and SIGHASH_SINGLE past the end of the output list wrote 32 zero bytes and continued – "BIP143 substitutes a zero hash in this position; BIP341 deliberately does not, and this file had carried the BIP143 behaviour over." Both were false accepts. Both were found by "asking Core for the answer to 19,721 vectors and comparing, which is the only method that finds a false accept at all."

And incident #25, BIP66/DERSIG: the flag was computed, threaded into the interpreter... and "nothing read it. Both checksig callbacks went straight to der_parse_sig, whose own header advertises that it is 'TOLERANT of non-minimal INTEGER encoding' – correct, and necessary, *below* the activation height. Above it this node accepted signatures Core rejects. Not a stall: a chain split, and unlike #22 (a false reject that stopped the replay loudly) this one is a false

accept, which splits silently." The audit of what was actually accepted was empirical, not armchair: "fuzzing 18,322 encodings against Core's own CheckSignatureEncoding... 4,643 of them were accepted here and rejected by Core."

THE SETcc INCIDENT, OR: ELEVEN OPCODES AND ONE BYTE WIDTH. Incident #28 is the single most instructive interpreter defect, because of how invisible it was. The interpreter computed flag-like values with SETcc – which writes *eight* bits into the destination – and then, in eleven numeric opcodes, pushed "the OPERAND" back onto the stack where it should have pushed the result, in a form that re-read stale high bits. Against 63,036 generated scripts compared to Core it diverged on 11,780 of them, "5,050 of them false ACCEPTS" – and not one of those 11,780 scripts exists in the historical chain, so the million-block replay ran clean across it. Five thousand chain-splitting divergences per 63k probes, discovered only by synthetic probing against Core.

THE EXCEPTION-BLOCK, WHICH THE CODE ALREADY KNEW ABOUT. Incident #22, the false REJECT at height 692,261, deserves mention because of what it says about living with Bitcoin's archaeology. That block is Core's Taproot script_flag_exception – "the single mainnet block that violates the taproot rules. It was mined well before activation at 709,632" – exempted by *hash*. "This project already had that right. script_flags_for_block implements the exception, and tests/test_script_flags covers it 13/13... The defect was that the two P2TR dispatch sites never consulted the flags they had already computed... The right answer was worked out and thrown away."

THE CENSUS, AND THE THREE LESSONS APPENDED TO IT. In the same week, the project did something almost unprecedented in software engineering: to stop discovering walls one at a time, it sampled 257 blocks of the *un-replayed* chain ahead of the replay via the oracle, classified every input's script shape, and cross-referenced the measured maxima against every hard-coded cap in the verifier – publishing, with evidence txids, where it expected to die. CHAIN_AHEAD_CENSUS.md predicted the first hard wall (the witness-item cap: 8, against a real chain maximum of 21) and then, when it was revised after the fact, recorded its own misses with brutal precision:

"The lesson. Conclusion 2 above – 'the remaining risk is coverage, not code' – was the wrong call, and it was wrong because the census classified shapes by whether a code path *existed*, not by whether it *worked*. A dispatch table entry that routes a valid opcode to .bad_opcode looks identical, from the outside, to a correct implementation... 'handled' in the table above should be read as 'has a code path', never as 'verified'."

"A second lesson... the shape that is *consensus-valid and therefore mineable* yet has simply never been mined. Incident #18's oversized

tapscript stack item is exactly that. No sampling density would have produced it; only reading the path against Core's ExecuteWitnessScript line by line did...

'What has the chain done' and 'what may the chain do' are different questions, and this document only ever asked the first."

"A third lesson... the sparse answer to 'how big do output scripts get' was 105 bytes, and it was wrong by a factor of nineteen... A census result is a statement about its sampling interval. When the number it produces is the reason not to fix something, re-sample at the density the conclusion needs before believing it."

Three documents, three blind spots, each named only after it had cost a defect: code that exists is not code that works; the chain's history is not the chain's possibility space; and a measurement is a claim about its sampling density. The census also, characteristically, shipped with the un-edited wrong predictions left in place – "The original text is left unedited – its value is as a record of what the method did and did not catch."

Chapter 6. The interpreter as archaeology

To verify Bitcoin's script language in 2026 is to maintain a small museum with live exhibits. The interpreter incidents read like a guided tour:

- `OP_NOP1` and `OP_NOP4..OP_NOP10` "were treated as bad opcodes" – the quiet

no-ops of BIP141-era cleanup, unreachable by any test that only used the famous opcodes.

- `OP_CHECKSIGVERIFY` (`0xad`) "routed to `.bad_opcode` under `SIGVERSION_TAPSCRIPT`."

BIP342 keeps `CHECKSIG` and `CHECKSIGVERIFY` (re-specified for schnorr) and disables only `CHECKMULTISIG(VERIFY)`. Real HTLC-style leaves (`<pk> OP_CHECKSIGVERIFY ... OP_1 OP_CSV`) died on the `CHECKSIGVERIFY` – incident #16, which is also the incident whose *first diagnosis was wrong in print*: the zeroed script-eval context was a real bug, but a too-permissive one, and "the fork's differential test overturned it." The rule written from it: "a confident inline root-cause is a hypothesis, not a finding... Handing the hypothesis to a differential check against a real fixture – fails on old code, passes on new – is what distinguished them."

- `OP_CHECKMULTISIG`'s `FindAndDelete` of signatures from the `scriptCode` ran

"unconditionally. Core does it only under `SigVersion::BASE`; under `WITNESS_V0` it corrupted the `scriptCode` and would have rejected every valid witness multisig."

- The sighash substrate under the interpreter had its own archaeology: a transaction mixing legacy and segwit inputs "is witness-serialized, so the legacy input's sighash must be computed over the witness-stripped form. First possible at 481825; it would have rejected most early-segwit-era transactions."

- And the museum keeps acquiring. When the corpus work came back clean on

1,128 mutated real spends, the same day's synthesis pass found "a second false accept, and a worse one": the BIP341 script-path commitment "compared only the tweaked output key's X coordinate and ignored the control block's low bit – the tweaked key's Y PARITY... flipping it on any otherwise-valid script-path spend produced a transaction this node ACCEPTED and Core REJECTED... a chain-split-direction false accept, reachable by flipping one bit of witness data, with no key material and no grinding." The exercise also exposed "three frozen taproot vectors that hard-coded parity 0xc0 without ever computing it: they were never valid spends, and passed only because the verifier ignored the bit." A test corpus can be complicit; when the vectors are frozen and the verifier is blind, both agree and nothing learns.

By the time the interpreter surface was done – 7,797 bare-script probes, all agreeing with Core, plus real-spend mutation corpora stratified across "pre-BIP16, P2SH, dersig/CSV, segwit v0 and taproot" – the project had assembled something rarer than a working interpreter: a machine-checked transcript of Bitcoin's script rules, one that any other implementation could be graded against. That artifact matters to the ecosystem independently of whether this particular node survives. It is the first thing this experiment gave back.

PART III: THE LEDGER ENGINE – storage as a contact sport

Chapter 7. The store that wrote the disk thirteen times

Every Bitcoin node eventually has to answer the same boring question: where do unspent transaction outputs live? Bitcoin Core answers it with LevelDB – a log-structured merge-tree behind a bounded cache – an architecture so ordinary it is invisible. This project arrived at the same answer the hard way, by first building the wrong thing and measuring how wrong it was.

The first UTXO store was an in-memory open-addressing hash table over an mmap'd, file-backed blob – sized upfront for the final answer (~408 million live UTXOs, 2^{30} slots for headroom, 51.5 GB). During a genesis-to-tip replay it performed terribly, and the log's diagnosis is a model of evidence-driven postmortem work:

```
"A full-archive UTXO-set-from-archive replay started at ~9,000 blocks/sec and collapsed to under 100 blocks/sec by 15% progress. Root cause, confirmed empirically (iostat, /proc/PID/io, /proc/PID/smmaps_rollup): the in-memory hash table has to be pre-sized upfront for the FINAL total live UTXO count... but during replay the hash function scatters writes essentially randomly across that entire 51.5GB mmap'd, file-backed structure from block 0 onward, regardless of how few entries are actually live – table measured being written back to disk ~13x on average (606GB actually written vs ~47GB of real touched data)."
```

Thirteenfold write amplification. The fix was not cleverness; it was archaeology-in-reverse – reinventing what LevelDB had been doing since 2011:

```
"Bitcoin Core avoids this with LevelDB (an LSM-tree) behind a bounded -dbcache, not one giant pre-sized structure. Built the equivalent from scratch (no external libraries, matching this project's ethos): bitcoin_utxo_lsm.asm, reusing bitcoin_utxo.asm's open-addressing table UNCHANGED as the memtable engine... instantiated small and fixed-size, never resized."
```

Phase 1: a bounded memtable, a per-generation write-ahead log, and sorted, Bloom-filtered immutable runs flushed when the memtable fills – "On-disk layout: utxo_manifest.dat (run-generation list, published via temp-file+fsync+rename -- NOT the old utxo.idx's 0_TRUNC-rewrite-in-place, which has its own unfixed crash hazard)." A detail worth pausing on: the AI cited, as motivation, a crash hazard in its own *previous* design, from days earlier. The project's memory was in the files.

Phase 2: streaming k-way-merge compaction, which produced two textbook distributed-systems bugs worth naming precisely, because they are the ones every engineer eventually meets:

- Key aliasing in the merge: the "advance every slot matching the winning key"

loop compared against the winning slot's own live key field, which mutates as it advances. "Fixed by snapshotting the winning key into a stable buffer before any advancing starts." The classic iterator-invalidation shape, wearing an asm costume.

- A register-confusion bug that corrupted the Bloom filter's byte array via

the wrong pointer: `mac_bloom_setbit`'s `bloom_base` landed in `rsi` instead of `rdx`, "while `rdx` still held the winning slot's OWN address from an earlier instruction... wrote into the winning slot's own header fields instead of the bloom bitmap, corrupting its `fd` (observed as an impossible `fd=2097163`), which then failed with a real I/O error the NEXT time that slot needed re-reading. Found via a hardware watchpoint on the corrupted slot's `fd` field, which caught the exact write instruction and its caller."

And a pre-existing bug found *while verifying the replacement*: the old store's WAL reload stored a length "via a 32-bit `mov` into an 8-byte stack slot then reloaded via a 64-bit `mov`, leaving stack garbage in the high 4 bytes" – the field-width rule showing up, as it always does, in the most load-bearing place.

The LSM paid for itself immediately: "Full-archive rate stayed flat instead of collapsing (e.g. 694 blk/s vs the old store's 79 blk/s at matching checkpoints)." An 8.8x improvement from deleting a bad idea, which is the cheapest performance work that ever exists.

Chapter 8. Crash-consistency, three times over

An LSM store with a WAL, a checkpoint, a manifest, and compaction is a small consensus system whose peers are crashes. This project fought that war on three separate fronts, and the order of discoveries is a syllabus in durability.

FRONT ONE: THE RESUME THAT RE-SPENT. Incident #5 – durable state ahead of the checkpoint, re-applied blocks rejected as double-spends – is told in Chapter 8; its lesson (checkpoints per unit of work, not opportunistically) is Chapter 8's conclusion. It earns its place in this chapter only because it set the pattern for the two that follow: the store was never wrong; the *claims about* the store were.

FRONT TWO: THE SIGKILL THAT WASN'T A BUG (incident #8, the `mmap` red herring). A storage optimization – caching run-file `mmaps` instead of re-opening them per lookup, "88% fewer syscalls" – was blamed for a resume failure it did not cause. The investigation could not reproduce anything against the `mmap` path; the real culprit was process supervision:

"The actual cause came from journalctl -u bmc-bitcoind.service: EVERY stop during a replay... ended in State 'final-sigterm' timed out. Killing ... SIGKILL at systemd's 90 s default. The replay is one utxo_live_catchup() call that runs for hours, and its per-block loop never read g_shutdown_requested."

"mmap was innocent; the resume path was the bug, and it had been since Stage D."

Two fixes, both general: the replay loop polls the shutdown flag per block and finishes its unit of work ("systemctl stop went from 90.23 s + SIGKILL to 10.05 s, measured"), and boot recovery reads the undo log – "undo_<applied +1>.dat existing means that block began and never checkpointed" – restoring captured prevouts and deleting created outputs before re-applying. The fix "then proved itself unplanned: the deploy's own stop was the last 90 s SIGKILL, it landed mid-block 343087, and boot logged RECOVERY: rolled back partially-applied block 343087 ... 147 prevout(s) restored and resumed with zero rejects."

The incident also left two permanent rules: first, the negative-control habit in full flower – "Three crash-injection scenarios incl. flush-mid-block give a set byte-identical to a never-crashed reference; the negative control reproduces the production line verbatim." Second, the rule born from the one mistake in the investigation: the coordinator destroyed the only reproducer by dropping the UTXO state while hunting the bug. "Mistake, recorded as a gate: snapshot utxo_/undo_ before any drop."

FRONT THREE: THE GHOST RUN (incident #41). Months' worth of hardening still left one window: a SIGKILL between durable writes and the checkpoint, followed by a restart that re-applies several blocks – "ghost" blocks whose undo data the apply path then destroyed:

"the killer -- apply_block_at blindly undo_discard()ed the height before applying, destroying the one piece of data that could reverse a ghost, an instant before the fresh apply rejected on the ghost's own already-spent inputs."

The final design makes every block-apply idempotent and makes boot roll back the whole contiguous ghost run descending, "because disconnect is LIFO; chained cross-block spends make the order load-bearing." The test forks crash scenarios; "Negative control against unfixed main reproduces the production failure exactly -- 7 FAILs old, 23/23 new. This retires the 'never interrupt a catch-up' operational hazard: kill/restart mid-rebuild now self-heals."

That last sentence is the quiet triumph of the whole storage arc: the system was not merely fixed; the *operational hazard* was deleted. The same transformation happens three more times in this report – the SIGKILL stop, the blind recovery, the checkpoint truncation window – and it is, arguably, what durable engineering is: each incident ending not with "patched" but with "this class of failure can no longer be expressed."

Chapter 9. When the counter lies, and when the set does

Storage correctness has a taxonomy this project learned bottom-up: there is state (the UTXO set), there are claims about state (counters, heartbeats, checkpoints), and there are claims about claims (verification tools, recovery loops). The disasters differ by level.

THE COUNTER THAT WENT NEGATIVE. The LSM's live-entry count was re-derived on reload as "the live entries of the current unflushed memtable generation ONLY" – the tens of millions of UTXOs in flushed runs were never counted, and every delete of a pre-existing key pushed the number down "until it crossed zero." A from-scratch build kept it accurate; only reload was broken, "which is why it had been meaningless since the very first resume and nobody noticed." The fix persists ground truth instead of re-deriving a guess, with a one-time recount over old manifests – and then, immediately, the honest counter caught a real bug: the rebuilt set was 22.2 million entries richer than Core's, because "live_on_output puts every output into the set with no script inspection at all, while Core never writes a provably-unspendable output to the chainstate." Verified "by magnitude rather than assumed: sampling the oracle every 25,000 blocks... extrapolating to ~21.7M nulldata outputs – the observed delta to within 2.5%." The sharpest detail:

"The codebase had already made this argument -- once, correctly, and only for a single output. `utxo_live.c:548` excludes the genesis coinbase precisely because applying it 'would leave this node one UTXO richer than Core forever'. The same sentence describes twenty-two million other outputs. Getting a rule right in one place is not the same as holding it as an invariant."

THE COUNTER THAT OVER-COUNTED BY 7.9 MILLION (incident #45). Later, during a ghost-heavy rebuild, the O(1) live counter over-reported by exactly 7,890,418 while – MuHash proof in hand – "the set itself was never wrong." Root cause: a kill landing between the flush's manifest write and its WAL truncate leaves a manifest whose persisted base already contains the WAL's ops next to a WAL that still holds them; the reload's counter arithmetic then double-counts. The fix is a piece of careful epistemics: base+tail arithmetic "is trusted ONLY when the tail is EMPTY (the clean-shutdown case); ANY non-empty tail takes the existing exact recount... paid only on unclean-shutdown boots." A cheap number may not override a possible lie; when it might, you pay for the measurement. The rule written from it: "A counter is not a measurement... when a cheap number and an expensive measurement disagree, the measurement is the number."

THE SET THAT RESURRECTED ITS SPENDS. The third level of failure is the one that makes this chapter the prelude to Part VI: a storage *lookup* that silently returns "not found" for a key that is present. Held for its proper home, but named here because it is the same LSM, the same counter culture, the same lesson in its most expensive form: 2,596 spent coins still live, "the false-accept/double-spend direction, in production, for hours, caught only by muhash parity against Core."

The chapter's summary, in the project's own rulebook:

"An incrementally-maintained counter is telemetry until reconciled against a ground-truth walk; any restore arithmetic whose correctness depends on a crash not having happened in a specific window is wrong by construction (detect the window, or pay for the recount)."

Chapter 10. The capstone

On 2026-08-25, at a quiesced height of 963,967, the project ran the test that justifies everything in Part II and Part III. The full replay – every block from genesis, every signature verified, no assumevalid – had produced a UTXO set. Bitcoin Core, running in parallel, had produced its own chainstate by its own code, its own storage, its own ordering, its own history of bugs. The two were hashed the same way Core hashes its set: MuHash3072, a 3072-bit multiplicative group hash whose digest over a set is insensitive to iteration order and sensitive to every byte of every entry – outpoint, value, height, coinbase flag, script.

"field Core oracle (gettxoutsetinfo muhash 963967) this node txouts 165,726,554 165,726,554 EXACT total_amount 2,007,466,988,462,591 sat same EXACT bogosize 12,980,678,786 same EXACT muhash 1e3c77ad25f40961f1f757a77960b7c49a5c7bd0 same IDENTICAL 91597bd925d561a5c202c118"

"The muhash equality is the cryptographic proof: every one of 165.7M UTXOs -- outpoint, value, height, coinbase flag, script -- byte-equal to Core's chainstate."

It is worth holding this fact up to the light. Somewhere in that digest are the height fields incident #29 had flagged as divergent – "two height fields on two outpoints from 2010" – and by capstone day even those had been rebuilt away, so this digest matched with "no filters, no overrides, no corrected fields." A ~50,000-line assembly consensus engine, a ~66,000-line C orchestration layer, and a home-grown LSM agreed, bit-for-bit, with the most reviewed financial code in existence, on the state of 165.7 million property records. Three weeks after the first SHA-256 test.

And the project, to its enormous credit, immediately printed the caveat that no marketing department in history has ever permitted itself:

"It establishes the ACCEPT direction about as strongly as it can be established... It establishes nothing about the REJECT direction, and this project keeps finding that the two differ."

Because by then they had the receipts – the SETcc bug, the BIP30 gap, the parity-bit bug – every one of them invisible to the replay that had just produced the perfect hash. This is Part II's lesson, restated at its sharpest: the MuHash is the strongest ACCEPT proof available short of proof-of-consensus,

and the false accept walks unbothered through every happy-path certification ever devised.

The instruments built for the capstone stayed. `utxo_setinfo` became "the tool of record" (a ~6-minute walk over 165M entries with a quiescence gate, "the daemon sits at its stopatheight ceiling, so the UTXO files are static -- verified with two stats five seconds apart before starting, rather than assumed"), and later the `coinstats` index folded MuHash incrementally per block, so `gettxoutsetinfo` answers "in ~33 ms instead of a full walk." Parity became continuous instead of ceremonial – which is the only form of correctness monitoring that survives contact with a running system, and is why, as Part VI will show, the resurrection of 2,596 coins was discovered in hours rather than never.

PART IV: THE HUNT FOR CYCLES – performance in the land of Amdahl

Chapter 11. The profile that started the war

By 2026-08-21 the replay was running at full historical depth, and it was slow. Not mysteriously – measurably. The first serious perf session began with a tooling problem that deserves recording because it is the kind that defeats most teams: the running binary had been rebuilt since the process loaded, so perf reported raw offsets against "bitcoind (deleted)". Ground truth was recovered by copying /proc/<pid>/exe, discovering that perf had printed *file offsets, not virtual addresses*, rebasing by 0x400000, and validating the correction against the nm table ("offset 0x368d8 rebases to exactly the fe_mul.zero label. 100% of the previously-unresolved userspace samples resolved"). Ten seconds of samples then told the project what to work on for the next four days:

```
"DSO split: bitcoind 60.2% · kernel 37.9% · libc 3.9%. ... secp256k1 field/scalar
arithmetic ≈ 52-55%... kernel file-read path ≈ 31%."
```

Thirty-one percent of all cycles in the kernel's file-read path. Not in the block archive reads – the profile showed no store_read_at-family symbols – but in the UTX0 store's LSM lookups, "once per transaction input" each, re-opening every run file and reading "the entire bloom filter (4 MiB at bulk scale) into TLS scratch in order to test exactly 3 bits." The fix – a per-thread cache of read-only mmaps, with the audited assembly path intact as fallback – cut the syscall census by 88% and, end to end, more than quadrupled replay throughput. One microbenchmark showed 47x; the project printed the caveat itself: "That microbench has small blooms and a warm page cache, so it is an upper bound; the syscall table above is the honest structural number."

The same session measured the gap that organized the crypto work: libsecp256k1 built from Core's own vendored source, same CPU, same moment, back-to-back with this project's bench:

```
"libsecp256k1 (Core v31.99 config) 21.8 µs/verify ≈ 45,900 verifies/s/core
bitcoinmachinecode ecdsa_verify 120.9 µs/verify 8,271 The crypto gap is 5.5x, not
the ~1.5-2.5x assumed earlier from public figures."
```

Five and a half times. The public figures people quote are measurements of someone's average day; the useful number is the one you take on your own box, in your own load, against your own code. (Two days later the gap was ~1.65x. The path there is this Part; the epigram is that most of it was not cleverness – it was algorithm substitution plus inlining plus one inversion.)

Chapter 12. $O(n^3)$: the biggest win was a data structure nobody had deleted

A re-profile at height ~617,000 inverted the project's mental model. The single largest symbol in the entire replay was `read_cs` – a compactsize reader – at 22.44% of all cycles, with `sw_seq` and `sw_prevout` behind it: "34% together, larger than the field multiply." The BIP143 sighash code was re-walking the transaction's input list from the beginning for every input, inside a loop over every input, called once per input:

```
"because segwit_v0_sighash is called once per executed OP_CHECKSIG, i.e. once per
input, the whole thing was  $O(n_{in}^3)$  per transaction. Core has never paid any of
this: PrecomputedTransactionData walks the transaction once."
```

The fix was one bounded pass recording the offset of every input and every output; "every accessor afterwards is an array index." And then the detail that makes this chapter worth reading twice: a `CTxOut`'s BIP143 serialization "is byte for byte its wire encoding, so there is nothing to build. `hashOutputs` is a sha256d over the contiguous slice [`out_off[0]`, `out_off[nout]`) in place." The re-serializer – the very function whose unbounded buffer had caused incident #21's stack overflow – was *deleted*, not optimized. When the canonical form of a structure is its own bytes, the compiler-friendly move is to stop compiling anything.

Measured on real blocks – 20 mainnet blocks at the exact profiled height, 53,400 witness inputs driven through the sighash: 5,735.90 ms → 209.00 ms. "107.41 us → 3.91 us per witness input, 27.4x." `read_cs` calls: 131.9x fewer. And then the discipline, which this entire Part obeys and which most performance cultures never learn:

```
"Projection, with the bound stated: the three symbols are 34.07% of profiled
cycles, so the Amdahl ceiling is  $1/(1-0.3407) = 1.517x$  and nothing here can beat
it... Do not believe that without re-profiling: section 7 exists because a
projection off a 227,000-block-stale profile said 1.40x and delivered 1.15x."
```

The same paper day, the BIP341 taproot sighash got the identical treatment (" $O(n_{in}^3)$ per transaction" again – the bug pattern had been cloned with the file): 18.81x on 19,870 real taproot inputs from five blocks; one block's 1,500-input consolidation went from 3.83 seconds of sighashing to 196 ms. The common case "is not pessimised" – checked four times with interleaved min-of-15 repeats and non-overlapping distributions, because the honest question about any asymptotic win is what it cost the 99% shape.

But the *method* is the harvest. Three laws fell out of the sighash campaign: (1) profile before optimizing, because two profiles three days apart ranked the same symbols in opposite orders; (2) state the Amdahl ceiling before writing the code, because "the projection was wrong by a factor of 1.2 on a stale share" and knowing the ceiling converts optimism into arithmetic; and (3) "the bounds must survive the rewrite" – every one of the incident-#21 fixes (bounded

readers, split-form overflow-proof bounds) had to be carried into the faster code deliberately, because performance rewrites re-introduce the memory bugs they fly past. When the taproot path's aggregate buffers moved to the faster design, an OOB read that needed no crafted input at all – "14,579 of the 36,712 faults needed no byte poisoning... plain truncations of real mainnet transactions" – was still sitting there, one dereference of `*end` from a peer. Faster code that is wrong faster is a chain split with better mileage.

Chapter 13. The multiply at the bottom of the world

With I/O and serialization gone, the replay was one thing: `fe_mul`, 55.9% of all cycles. "The replay is now almost purely bound by the field multiply." What follows is a case study in doing optimization research the way it should be done – measure the ceiling, not the folklore.

The first probe asked what the hardware can even do. A hand-written `uarch.asm` microbenchmark established the machine (5.478 GHz achieved under live load; `adcx+adox` two chains at 2/cycle) before any code changed. Then a `mulx`-substitution attempt – the safe one – delivered "82.5 -> 83.7 Mops/s, ~1.015x," a result the log refuses to bury: the bottleneck "wasn't touched by swapping just the multiply instruction." The actual rewrite split the four partial-product rows of `fe_mul` onto two independent carry chains (`adcx` on CF, `adox` on OF – "two rows using DIFFERENT flags share no dependency, so the CPU can overlap e.g. row 1's work with row 0's still-retiring chain (the actual mechanism these speedups come from; attempt 1 kept everything on one serial chain)"), verified first – "150,000,000 random trials + edge cases... zero mismatches vs the original, zero mismatches vs the independent reference" – then *measured*: 1.13x, against an expected 20-25%. The diagnosis of the shortfall is better than the win:

"isolating the row computation alone shows it really did get ~44% faster ... but the separate merge pass this design needs (rows write to independent buffers rather than accumulating in place) costs ~0.078s, consuming roughly 58% of the raw row-computation savings. The clean row independence that made the parallel chains safe to reason about is also what gives most of the gain back."

Then the inversions. "There are exactly two inversions per `ecdsa_verify`" – each a Fermat exponentiation walking 256 bits with 503 `fe_mul` calls because `p-2` has 247 set bits. The classical $x_k = a^{(2^k-1)}$ addition chain reaches the same exponent "in 255 squarings and 15 multiplies." One inversion: 4,712 ns → ~2,091 ns, 17% of a verify recovered. The guard rail against folklore again: a variable-time binary `xgcd` exists in the tree and "measures 3,590 ns... *slower than the 2,091 ns the addition chain now costs*" – the textbook "faster" algorithm was slower *here*, and `safegcd`'s further ~1.1 μs was priced at "~600 lines of new consensus-critical primitive. That trade is not worth taking today and is written down here so the next session does not rediscover it."

Then GLV – the secp256k1 endomorphism splitting the scalar multiply into two smaller ones, with $w=5$ wNAF windows – cutting point multiplies from ~252 doublings to ~128: "GLV end-to-end 5.68x at 379k-399k vs baseline (4.39x before GLV)" on baseline-matched height ranges. The end-to-end number the project published for the whole 4.1+4.2+4.3 campaign:

```
"ecdsa_verify: 115-121 µs -> ~39 µs (loaded: 44) vs libsecp256k1: 5.2x slower ->
1.65x slower kernel (I/O) share: 31-38% -> 5% replay, identical heights: 7.8 blk/s
-> 34.1 blk/s (4.39x); with GLV: 3.9 -> 22.2 blk/s (5.68x)"
```

(The 1.65x belongs to this campaign – I/O, inversions, GLV. The ~1.2x that Part IV later quotes is a later sequel: the single-pass sighash and field- inlining work closed the remainder from the other side. The µs figure and the later ratio are different episodes of the same descent; printed side by side they read as a contradiction, which is why this report says so here rather than leaving the reconciliation to the reader.)

Then the negative results, which this project treats as first-class deliverables. The obvious next lever – adopt libsecp256k1's 5×52 limb representation with lazy reduction – was BUILT, tested at depth, and REJECTED with numbers: "the *correct* implementation measures 1.07x, because `fe_add/fe_sub` each need an extra correction round once `fe_mul` may return $[0, 2^{256})$, and there are more of them (1,243 + 1,436) than multiplies (2,262)." And the investigation caught four latent consensus hazards on the way: `point_add` selects its doubling branch by *limb equality*, "a non-canonical operand there returns infinity instead of $2P$, silently." The real lesson of 5×52 arrived sideways: what actually wins in libsecp256k1 "was found while instrumenting, worth more" – the insight that libsecp's structural advantage "is not the representation, it is that libsecp256k1's field ops are *inlined into the EC formulas*, so consecutive values stay in registers." The 4×64 code got that property directly: 58 call sites to `fe_add/fe_sub`, 57 eliminated into 40 register chains, "17 whole field elements' worth of store-then-reload" deleted, no algebraic shortcuts – "this banks no algebraic shortcut. What disappears is 57 call/ret pairs." Result: 1.10x on `ecdsa_verify`, 1.22x on `point_double`. The highest-value crypto optimization of the whole project was, in essence, a decision to inline functions – reached only by measurement, after rejecting the copy-the-famous-library plan on evidence.

And AVX-512 IFMA: evaluated honestly, "a real 8-way kernel was then built and validated, not extrapolated... 4.11x on the field multiply, and the hardware objection does not apply" – then deferred with a cost analysis that is a model of the genre: every consumer must be rewritten 8-wide (a second complete EC stack on the consensus path), the table reads need ~80 vector ops per windowed add, and "the batching point does not exist yet for most of the remaining work" because legacy `OP_CHECKSIG`'s result steers script control flow synchronously. Amdahl stated at the end: "if the whole 74% crypto share went 4x, Amdahl gives... 2.3x on replay – real, but a second EC implementation's worth of consensus risk for it." A working 4x-speedup prototype, killed by arithmetic

and honesty. The trap on the way is also preserved: the first vector kernel measured 3.2x worse because gcc kept the accumulators on the stack; only checking the disassembly prevented a false verdict against the hardware. "The measurement is only valid because the disassembly was checked."

Chapter 14. The bottleneck is not arithmetic, it is serialization

On 2026-08-23 at 02:30, after four days of micro-optimizing the field multiply, the project noticed its verification worker pool was decorative:

```
"thread states, verification worker: 32 sleeping, 1 running effective parallelism:
~1 core of 32"
```

The workers *skipped taproot on purpose* – a code comment said so ("taproot pass 3 stays sequential/single-tx-at-a-time") – because the taproot helper buffers lived in .bss, process-global scratch whose own header documented the assumption: "Global scratch... used by the small helpers is single-threaded."

```
"This is the third time tonight a written-down assumption turned out to be a
scheduled outage; incidents #22 and #26 were both preceded by an explicit 'known
divergence, deliberately left' note. A documented assumption needs a test that
fails when it stops being true, or it needs fixing."
```

That is one of the two or three most general lessons in this report, and it is not about assembly: an assumption written in a comment is an untested dependency. Every team has its taproot comment – the sentence that quietly sets the parallelism ceiling, the latency budget, the scale limit, the "temporary" workaround that the architecture has silently grown around. The project's fix was to make the assumption false (thread-local scratch, proven by a stress test that caught "29,236 wrong digests of 96,000" against the globals) and then parallelize the taproot pass across 32 workers: measured 2.72 → 26.6 effective cores, 7.49x on the verify phase of real blocks. The four-day micro-optimization campaign had been, in wall-clock terms, polishing one core of a 32-core machine while the design serialized on a global variable. Amdahl is not only about instruction shares. It is also about the processes you forgot to parallelize – and about measuring that before, not after, the `fe_mul` work. (The log's own ordering note, §14.3, says exactly this: "parallelising the taproot pass comes before any further `fe_mul` work. A 1.15x on 54% of one core is worth far less than moving that work onto 32.")

A quieter companion finding from the same profile: the constant-time question, resolved. `test_scalarmul_ct`, the timing-leak guard, had been failing on a loaded box – and the same binaries, at load 60, produced readings in *both* arms (0.952, 0.956) that matched the exact signature the test's tolerance exists to catch. At load 3.3, everything was clean. The lesson: "A calibrated instrument is calibrated for an environment... Run it outside them and it manufactures both false alarms and false confidence." Metrology, like code, has environments; a threshold is a claim about the conditions of its measurement.

Chapter 15. The GPU that was never needed

There was a CUDA phase. It ran roughly four hours (asm/cuda/WORKING.md is timestamped in a single commit stream), and its analysis is worth more than most teams' month-long GPU investigations:

```
"1,000,000 independent sha256d: CPU (asm SHA-NI) 8.57 Mh/s vs GPU (scalar CUDA,
unoptimized) 151.4 Mh/s – 17.7x. 1,000 sha256d: 3.35x. 100 sha256d: 0.60x."
```

The crossover – a few hundred hashes – is the whole GPU story in one number. "The GPU's entire value is throughput on batches, not latency on singles," and a node's hashing is a stream of small batches interleaved with serial control flow. The honest scoring table rates block-body validation's batched txid/merkle tier as the only real fit, ships the auto-detecting, fallback-safe dispatcher ("uses the GPU only when present AND the batch is large enough to amortize launch/copy, otherwise falls back bit-exact to the proven assembly crypto"), and – this is the line – wires it to nothing. The dispatcher is complete and the integration is a documented seam, because the product's actual bottleneck (Part IV's running theme) is a 55%-of-cycles field multiply on a latency-sensitive serial path, which a GPU cannot touch without the deferred-verify restructure the EC side doesn't have yet. The GPU work is not a failure; it is an option, correctly priced and correctly left on the shelf. Most organizations never learn to tell those two results apart from a defeat.

PART V: THE NETWORK STRIKES BACK – P2P, keep-up, and three bad days

Chapter 16. "It only advances when restarted" – incident #33

The node had been declared, more than once, a live mainnet participant. On 2026-08-24 that claim was audited by reality. The node sat at height 963,775 for fourteen hours and twenty-six minutes while the network moved eighty blocks ahead – with peers=8/8 in every heartbeat. A restart pulled all eighty in twenty-two seconds.

The first lesson is about evidence of health:

"Every tip advance during the previous night came from the BOOT CATCH-UP path at a restart, and I restarted the daemon repeatedly to deploy fixes. '8/8 peers plus an advancing tip' reads exactly like keep-up. It was the restarts."

The second is about silence:

```
long ok = node_sync_multi(fd, store, loc, nloc, buf, cap, &cnt); if(ok != 1 || cnt
<= 0){ anchor_locator(...); return 0; } / no log /
```

"ok != 1 (the exchange FAILED) and ok == 1, cnt == 0 (the peer had nothing, normal at tip) exit through the same silent return. In a log that prints a line for every dial, drop, replacement and heartbeat, the one condition that mattered printed nothing at all."

The root cause was the most ordinary kind of bug: a parameter in the wrong units. `cons_verify`'s fourth argument is the txid-scratch capacity *in txids*; the steady-state sync path passed 64.

"node_sync_multi could not validate any block with more than 64 transactions. Height ~51,726 is where mainnet blocks first cross that, and every block at the chain tip has thousands -- so at the tip this path fails 100% of the time."

The reproduction line tells the story better than any diagnosis: "pass 1: node_sync_multi ok=0 cnt=51727." It downloaded, validated, and stored 51,727 blocks – then reported failure, because the caller tested `ok != 1` and threw the work away. The sync had been *working* for hours at 800 blocks/s; only its return value was a lie, and the one condition that could see the truth was the one with no log line. After the fix (scratch to `.bss`, cap 80,000 – "MAX_BLOCK_SERIALIZED_SIZE/60 ~= 66,666 txids"), the node followed two consecutive blocks with no restart, and tx=3923 in a store line proved the old cap had been rejecting every modern block.

Chapter 17. The units audit, and the four cap bugs

"One wrong cap implies the units are easy to get wrong," the project wrote, "so we audited every call site." Incident #34 found three more in the same function family – two of them far worse, because a cap that is too small does not merely reject, it licenses writes past a buffer. `node_drain` passed cap 256 into 1,024 bytes of stack scratch (room for 32 txids): "a block with more than 32 transactions writes up to 7 KB past it, over the function's own locals, saved registers and return address. Every block at the tip has thousands." `node_serve_loop` passed $(2000 \times 81 + 8)$ – a headers-page *byte size* – into the *count* parameter, licensing 5.18 MB of writes into a 162 KB buffer that its own headers page was sharing. All three moved to dedicated .bss scratches with a matching cap. The generalization, which belongs in every code-review checklist:

"a function whose parameter is a COUNT and whose caller owns the BUFFER is a units trap, and this codebase now has four instances of it in one family. The C callers all get it right for a structural reason -- they write `sizeof scratch / 32`, which cannot disagree with the buffer. The asm callers each wrote a literal."

The literal-vs-derived distinction is the whole lesson. The safe form existed in the same tree; it was not reused. Incidents #36, #37 and #38 – the overflow-audit cluster – are the same finding in a different costume: length bounds written as `p + attacker_length > end` wrap for attacker-chosen values near 2^{64} , "and this codebase wrote it that way in EIGHT places while the safe split form sat in the same tree. The lesson is not 'fix these eight' but 'the safe form must be the only form.'" Note what #36 also did to its own author's theory: the first reasoned claim was that the whole `>MAX_SIZE` range diverged from Core; building the test showed only four wrap values actually slip through. "Proving beat the theorising again: the divergence is exactly 4 values, not a 4 GB range."

Chapter 18. `connect=`, or the config option that lied

Incident #35 cost the project its first honest end-to-end benchmark. The harness set `connect=192.0.2.1` (a TEST-NET address, guaranteed unroutable) to isolate the replay from the network. The node printed

```
[boot] connect= set -- skipping all peer discovery [dl] no discovered peers;
temporary seed fallback [dl] outbound 0 = seed.bitcoin.sipa.be ...
```

and proceeded to append 567 blocks past the benchmark's truncation point – invalidating the run – while failing seed legs starved the replay to zero blocks in eight minutes. The degraded-recovery path never consulted the `connect-only` flag: "Core's `-connect` means 'these are the ONLY peers'; the correct degraded state under it is NO outbound peers, not 'substitute the seeds.'" The log's verdict is the sentence that should be taped above every config-system design review:

"a setting that is honoured on the main path and quietly bypassed on the fallback path is worse than one that does not exist, because the log says it was applied."

This defect had a systemic cure a week later, and it is the best product of the whole audit cycle: an explicit list of every Core option this node does not implement, each one named at startup –

```
[config] whitebind= is a Bitcoin Core option this node does not implement -- it
has NO EFFECT
```

– retiring the failure mode the codebase had reproduced repeatedly (externalip parsed and never read; maxreceivebuffer parsed and read nowhere; whitelist= sitting in a live config doing nothing). "A test asserts the list and the implementation move together in both directions."

Chapter 19. The serve path: two masks, three complicit tests

The most instructive P2P incident is the one about *inbound* serving – because it explains why a whole test suite can be innocent bystanders. "FEATURE_GAPS carried one unverified line... Checking it found two independent defects, either of which alone made inbound block serving impossible. Together they masked each other, which is why the first fix appeared to change nothing."

Cause 1, the byte-order relapse: index.dat holds wire (raw sha256d) order; the boot loader byte-reversed every record on the belief the file held display order, so the hash index was keyed backwards and "a getdata for a block we hold fell through and the peer got nothing, not even a notfound." The forensic detail is the test autopsy:

```
"WHY THREE TESTS MISSED IT... test_serve built its OWN index with a plain idx_put
loop (no reversal), so in the test the table was wire-keyed and the serve loop
worked -- it proved the serve loop correct against an index nobody builds that
way. bench_hashidx compared the asm loader against a C reference that reversed
IDENTICALLY, so the two agreed with each other and were both keyed backwards.
test_truncate reversed every hash before looking it up, matching the loader and no
consumer. Each test encoded the code's belief rather than the file's contents."
```

This is the `fe_inv` exponent-table lesson (Part I) returning at production scale: tests built from the implementation's assumptions certify the assumptions. Cause 2: every inbound connection forked a child that reopened the UTXO store – 60–83 seconds on the real 165M-entry set – and said nothing for the entire window, longer than any real peer's patience. The fix was borrowed, not invented: pre-fork init once, children inherit copy-on-write, "Bitcoin Core does the same... never re-opens the UTXO per connection." Measured: 63 s to first serve, before; under a second after.

Even the hunt for this had a human-instrument lesson queued: the `h=-1` that made real index hits look like misses was the author's own probe "reading the height variable in the same printf argument list as the call that sets it – C does not

order argument evaluation." Suspecting your instrument before suspecting the code is the move that stopped a wild-goose chase of innocent recent changes.

Chapter 20. Relay, the orphan pool, and the mempool freeze at 4096

The transaction-relay work of 2026-08-26/27 is worth summarizing for its shape, not its every line: a receive-only node became a relay hop (orphan pool that parks missing-parent children, fetches parents witness-typed, and cascades them in; re-announcement with never-back-to-source discipline; a getdata service that answers misses with notfound "so a peer's in-flight tracking is never left hanging"). The keystone decision, though, was architectural: "mempool admission no longer has its own partial script verifier... tx_accept now calls THAT" – the same consensus verifier the block path uses, with a resolver seam over confirmed-set-plus-mempool-parents. "One engine, not two" closed, in one change, the legacy-spend gap, the taproot-key-path-only gap, the unconfirmed-parent gap, and the tip-anchored-maturity gap. Every fork of a verifier is a future divergence inventory; this project proved it knew that by *not* having one.

Then the mempool froze production "at exactly 4096" – the fixed-size policy graph was full, and the pool, unlike Core's, had no eviction; it just refused. The hotfix resized the graph; the real fix was behavioral: TrimToSize feerate eviction, a compaction pass to reclaim blob space ("eviction freed a slot but no space and the retry still failed"), a dynamic mempoolminfee that rises under congestion, and every Core-exposed relay limit wired to the config. A freeze at a power of two is such a beautiful, unambiguous signature of a hard cap – and the log records it as one more entry in the honesty ledger: an incident that "existed in production first."

The package-relay sprint has the report's three-best lines about differential testing packed into one session. (1) "The negative control matters here: with the reconsiderable class disabled, six checks fail. A test that passes both ways tests nothing." (2) "replaced-transactions is TOP LEVEL in submitpackage... Reading Core's schema instead of guessing it is the only reason that was right." (3) "the differential caught the same class of miss a third time: our TRUC rules did not fire inside a package AT ALL... That is three separate defects this session that only a real Core found -- the hermetic tests were all green each time."

Chapter 21. The privacy, anonymity, and misbehaviour arc

Two things in the networking record deserve the spotlight because they are where an AI-written node met *social* protocol, not just wire protocol.

First, the proxy. "Setting -proxy routed CONNECTIONS through the proxy while the node kept resolving peer hostnames with the system resolver: a plaintext DNS query naming every peer it was about to contact, from the same host, at the same moment. That is the correlation running behind Tor exists to prevent, and

it was intact." The fix set (names handed to the proxy; DNS seeds skipped under proxy; .onion/.b32.i2p resolution refused outright) arrived with a live demonstration, not an argument: on testnet4, the log line "not querying the DNS seeds: a proxy is configured." A feature is a set of consequences; the first implementation had implemented the connections and missed the point.

Second, the misbehaviour machinery. The 2026-08-29 audit's Finding 7 is the purest statement of a failure mode this whole experiment studies: "a repo-wide search shows `peer_misbehaving()` has zero call sites. The comment 'a peer could send malformed message after malformed message and the node would keep talking to it' is still literally true – the machinery exists, nothing drives it." The response wired one real caller (oversize message announcement, scored at the full threshold), then stated, in the response document, exactly what remains unwired (duplicate-inv spam, malformed payloads, handshake failures) rather than claiming the category closed. And the companion finding mattered as much: the misbehaviour table was process-local while the serve loop runs forked per connection, so "a peer that reconnects between offences is not tracked" – a defence that is per-child is a defence against a single-process adversary. By 09-02 the scores had moved to a shared file-backed ban table.

There is also the incident that should open every AI-operations handbook: the pre-deploy adversarial review before the anonymity-networks deploy. "The Tor/I2P/CJDNS work was merged, gated at 208 suites, pushed, and about to be deployed... A three-lens adversarial review of the diff found seven defects first. Two of them would have taken the production node to ZERO OUTBOUND PEERS on the next restart, and the whole test suite was green because nothing covered the path they were on." A reviewer proved the dial-pool bug by compiling a harness against the REAL peers.dat: "inet_pton and getaddrinfo fail on all 64 pool entries." The moral, in the log's words: "a change that touches a shared representation -- here, what a 'pool entry' is -- breaks consumers that no test exercises, and the suite's greenness says nothing about them. Both times the review found a defect worse than the gap the work set out to close."

PART VI: THE LEDGER LIES ONCE, TWICE – the 2026-09-01 double incident

Nothing in the previous five Parts prepared either the authors or the reader for what happened in the last forty hours of the recorded development. Three things occurred on 2026-09-01 that each, in a normal project, would have been the headline event of the quarter: the host was hard-frozen by a one-character bug in an analysis command; the live node's UTX0 set was found to contain 2,596 already-spent coins worth 5,589.97 BTC; and a boot header-sync answer from genesis corrupted the derived index files and re-downloaded 11,516 blocks under fake heights. All three are worth entire chapters, and they share a single epistemological root, which is why this Part is the report's center of gravity.

Chapter 22. The genesis-first answer

The incident opened with a log line the operator spotted at 11:21 UTC – a progress line, not an error:

```
[dlc] == elapsed 0:01:30 | overall: 968673/1931687 stored (50.15% of real tip) |
282 holes in [0,968954] reached so far (99.97% gap-free) ==
```

"50.15% of real tip." The node had been at tip. What happened: the boot's header mirror had fallen 12 blocks behind the archive (a design flaw of its own – the worker's leg sync stores blocks without appending headers to the mirror), and the boot header fetch sent that stale tip as a *single-hash locator*. Any peer that doesn't know that hash – a behind peer, an IBD peer, a peer on another chain – "legitimately answers from its genesis." The peer did. And the node accepted the answer:

```
"node_ibd_headers pages through the reply and appends 966,669 headers -- mainnet
blocks 1, 2, 3, ... -- at positions 965,018 and up, checking only that consecutive
received headers link to each other, never that the first one links to the block
we asked from."
```

Nine hundred sixty-six thousand headers later, the catch-up had extended index.dat with 966,657 records (92.7 MB of holes), started sixteen workers to "fill" them, and 11,516 real early blocks – fetched honestly from honest peers by the fake hashes – were appended into the archive's tail slack under garbage index entries. The lesson, boxed:

```
"Trust nothing a peer sends as a continuation of state you hold unless it provably
connects to that state. The block path always had this property (PoW + prev-hash +
consensus); the header mirror did not, because it was 'just a mirror.' Derived
files must be derived. headers.dat is recomputable from the archive; anything
recomputable should be topped up from the source of truth, not from the network."
```

The recovery is disciplined, and worth summarizing because each repair step taught something. Before touching anything: copy the three corrupted derived files to an incident directory ("Keep copies before repairing... and verify before declaring"). Repair build p trimmed only the all-zero tail – missing the 11,516 junk records because they carry real hashes. Build q trimmed correctly and then SEGWed: the new mirror top-up called `store_read_at` "which returns the whole block" into a 128-byte stack buffer. Build r fixed the buffer and SEGWed on a different violated contract: `store_get_tip` declared in C with one argument; the assembly takes two. Both were contracts "that the C side had wrong" on a path never previously exercised – the asm/C boundary's call-tax, again. Build s booted clean, and its first act of business was the new linkage guard firing against another peer: "headers from [a live peer] do not connect to our tip – discarding 3 header(s)." The repair validated itself within seconds. Then build t – an unrelated wallet change – got a genesis-first answer *again*, from two peers in a row, and the log's autopsy of why is the moment this incident becomes a law: "with the mirror now current, the boot asks from the newest block; a peer a block behind does not know it, and a one-hash locator gives it nothing else to match – genesis answers are *common*, not rare." The final fix rebuilt the boot header fetch in C with an exponential locator and per-page linkage/overlap/fork checks – Core's design, arrived at by being attacked by the network Core designed against.

And the collateral damage is its own lesson. During the repair, one stop landed between the checkpoint writer's truncate and its rewrite: `utxo.idx` was found empty on the next boot, and the node began a full from-genesis replay – hours of work. The fix (`write utxo.idx.tmp`, `fsync`, `rename`) is the oldest trick in durable storage, newly learned by this project, "the long-known 'SIGKILL/checkpoint window' is closed." A destructive write that is not atomic is a data-loss event with your name on it, whether it happens at 2 a.m. under SIGKILL or at noon under a careful operator's `systemctl stop`.

Chapter 23. The 2,596 resurrected coins

The from-genesis rebuild that the header incident forced (see above) completed on 09-01 with a muhash that did not match Core's: +2,596 txouts, +5,589.97458543 BTC. The response document is the most careful piece of forensics in the tree; its summary table is a horror story told in reporter-voice:

"At every height $\geq 539,017$ our set = Core's set + exactly 2,596 outpoints. Zero outpoints missing. Sum of the extras = the `gettxoutsetinfo` amount delta to the satoshi."

An unspent coin is, to a validator, a promise that a double-spend will be accepted. For several hours a live mainnet node carried a chainstate that would, in the audit's words, "have accepted double-spends" – of precisely these coins, which the chain had already spent – detected only because muhash parity

against Core had been made a continuous habit. The origin hunt is a model of cheap bisection (our logs print live-count at every height; Core's `coinstatsindex` answers for any height; the surplus window 340,578–618,297 fell out of one query per logged height), and it landed on eight log stanzas, one per memtable flush:

```
[utxo_live] REJECT h=539017 tx=5: input references a missing/already-spent UTXO
[utxo_live] FATAL: apply_block failed at height 539017 -- stopping catch-up [dl]
utxo_live_catchup FAILED at height 539016 -- attempting in-place recovery
[utxo_live] recover: compact manifest_n=2 -> 1 (result=1) [dl] utxo recovery
SUCCEEDED (1 compaction round(s)) -- tracking continues
```

The root cause, pinned the next day by a regtest reproduction, has three layers, and each layer is its own lesson.

LAYER 1 – the trigger: one day's perf work. A buffered memtable flush had moved the on-disk write offsets, but the sparse-index samples still used `lseek(SEEK_CUR)`, "which excludes the buffered bytes." Because a record is written as three pieces (key, value, script) with mixed 22–34-byte scripts, the buffer drained mid-record and "every sample after it points into the middle of a record: the read-side scan parses garbage and 10–15% of point lookups through such a run MISS." (Note the shape-dependence that hid it from the first repro attempts: fixed-size short records never straddle the drain.) Lesson: optimizations that change write *offsets* are correctness changes; the index side of a file format is a second writer that must be told.

LAYER 2 – the mechanism: an error path that was a trap. Every spend captures its prevout for the undo log via `get-then-del`. After a flush landed mid-block, the capture's lookup *missed* for run-resident coins – and the absent-coin branch, written back when re-applies were always rejected by verification anyway, read 0 as "already absent – a crash-resumed re-apply" and SKIPPED the spend. The coin stayed live. The next block then rejected on the same lying lookup – and the blind recovery compaction rewrote the runs with correct offsets, so the retry passed, leaving no visible failure and a permanent set divergence. "The store never lost a record... Only point lookups lied, and one caller believed them." That sentence is the report's epitaph for defensive programming: a 0 is fine here branch is a standing invitation to every future code path that can produce a false zero. The fix, deployed within a day: "an absent coin at apply time is a store lookup inconsistency: the block fails, the partial apply is rolled back without trusting a lookup, UTXO tracking HALTS (sticky, heartbeat marker) and the worker stops retrying with an operator message." Fail closed, loudly, and make the halt expensive to ignore.

LAYER 3 – the mask: recovery that reports success. This is the incident's real thesis, and the project had been warning itself about it since incident #1 – "it cannot distinguish 'manifest full' from 'genuine consensus rejection', a still-present design pattern worth treating any future FATAL+recovery pair with suspicion over." The blind compact-and-retry had been live for two weeks. It converted eight consensus-relevant anomalies into eight green "recovery

SUCCEEDED" lines. The new recovery is gated and *verified*: failures are classified (consensus reject / store error / archive), compaction happens only for a store error with a full manifest, a consensus reject backs off from the checkpoint "without touching the runs," and after any compaction the recovery walks the entire set and requires `walk == counter == pre-recovery count` – "else UTXO tracking HALTS... (sticky; heartbeat shows [UTXO HALTED ...])." The lessons list closes it: "A recovery that 'succeeds' is a claim, not a proof."

The repair itself – a purpose-built offline tool, verify-then-delete of each of the 2,596 outpoints with the sum checked to the satoshi, coinstats re-seeded from a full walk, muhash re-identical with Core at height 965085 within ninety minutes of the analysis landing – earns the incident a strange verdict: it is simultaneously the most serious correctness event in the project's history (a false-accept-shaped defect in production, exactly what Parts II and V said was invisible to every other test) and the best argument in this report for the project's verification culture. Nothing else on the tree could have caught it. MuHash parity did. The defect's own report says so, and the 2026-09-02 audit says so: "caught only by muhash parity against Core." Continuous verification is not overhead. It is the only sensor that sees in this particular dark.

Chapter 24. The awk command that froze the host

The muhash hunt itself ended, at 20:00:33 UTC, with the journal stopping. No OOM-kill was ever logged; the kernel "never got that far: with all swap consumed and 2 GB of reclaimable cache left it thrashed the file-backed working set (the daemon's mmapped, every binary) until nothing could run." The host required a hard reset; every session, the oracle's catch-up, and a guarded deploy-watch died with it.

The trigger, in its entirety:

```
awk 'NR==FNR{k[$1" "$2]=1; next} (k[$1" "$2]){print $4}' only_ours.keys bmc-diff-ours.sorted
```

"In awk, `k[x]` in an expression is not a membership test: it inserts `x` with an empty value. The first file (20,561 keys) populated the array as intended; the second file then inserted every one of its 165.7 million lines... ~62 GB against a 60 GB box whose swap was already full."

Measured after the fact on a slice: 390 bytes per line with the bad idiom, 8 MB total with `(key in k)`. The box had been running with swap 100% full for hours – a state the project *knew* it kept – and there was no memory cap on an ad-hoc pass over a 13.5 GB file. The lessons are written in the dry style of people defusing their own bomb:

"awk membership is `(key in arr)`, never `(arr[key])`. One character cost the box. Every one-off pass over a multi-GB file runs under a cap – `ulimit -v` or `systemd-run -p MemoryMax=` – and only after looking at `free -h`."

And buried in the report is the incident's quiet sequel: the first comparison of the two key dumps had used `sort -k1,1 -k2,2n` while `comm` requires plain byte order, so the set-difference itself was wrong in both directions – 20,561/28,415 "differences" of which only 2,596/0 were real. Both analysis bugs (`awk`, collation) had to be corrected before the real 2,596 could be trusted. The 09-02 audit's summary of the whole affair is worth quoting twice, because it is the technical and the philosophical reading of the same event at once: "the 2026-09-01 UTX0 incident produced 2,596 resurrected spends – the node briefly accepted coins that were already spent... in production, for hours, caught only by muhash parity against Core. The fix (halt-on-absent-coin) is correct, fail-closed, and verified live – but this is the same structural property [as the hand-written assembly], now proven in a running node."

PART VII: DEALING WITH YOUR HUMAN – the operator as an API, and as a mirror

Every other Part of this report is about the machine's errors. This Part is about the collaboration between the machine that wrote the code and the human who ran the experiment – because the records show that the relationship was itself an engineering artifact, with an interface, failure modes, and lessons. It is also, unavoidably, a section about what a human is *for* when the typist never sleeps.

Chapter 25. The contract: three modes of authorization

The project's worklogs record an explicit, evolving delegation protocol between "the user" (a single human operator, referenced throughout the logs in the first person plural of a two-member team) and the AI implementing agent. Reading the logs across three weeks, three distinct modes appear, and the AI appears to have been told, or to have learned, to classify its own situation before acting:

MODE 1 – DIRECTION. The human sets goals and constraints; the machine implements. Most of the log is this mode: "Directive: mempool policy identical to Core. Survey first." "DATADIR ISOLATION (operator requirement): Core's layout exactly." The user asked for the versioning single-source-of-truth work ("the user asked for both the application version AND the protocol version to come from one canonical definition"), and got a design document's worth of derived-constant engineering in response. Directives were also negative: "Wallet deferred per directive." The human's job in this mode is choosing the fight; the record suggests they chose fights the machine could not win by cutting corners, because every directive came with verification attached.

MODE 2 – CONFIRMATION. Anything destructive or externally visible while the human was awake required an explicit yes. The worklog preserves the primitive itself: "User approved (AskUserQuestion) a clean archive_drop_utxo_state() rebuild before redeploying, since the bug could have left latent corruption in the on-disk manifest... the block archive itself was untouched." Again for a second rebuild two sessions later. Again, days later, for the incident #46 offline store repair: "REMEDY (user-authorized): stop; a one-off tool dropped exactly the one index record via store_truncate_index_only... ~13 minutes end to end, no data loss." The human's yes was earned by the machine's habit of stating the precondition first – the repair note records even that *store_truncate_to's monotonic safety gate correctly REFUSED first*. The refusal-before-the-yes is what made the yes cheap.

MODE 3 – STANDING AUTHORIZATION. The overnight mode, and the most revealing document in the whole tree about the human-AI contract. Verbatim, from the 2026-08-20 worklog:

```
"User went to bed with standing authorization: 'If you notice any problems overnight, just fix them, and redeploy the server if you need to. Reindexing from the start is fine as we're still in active development.'"
```

What happened next is logged with a care that is itself the story. Incidents #1 and #2 – the two destructive-state incidents – each got an explicit user confirmation while the user was awake. Incident #3, at 3 a.m., did not: the AI dropped the on-disk UTX0 state and redeployed "WITHOUT asking first this time, squarely covered by the standing autonomous authorization (incidents #1/#2 each got an explicit user confirmation while the user was awake; this one happened after they had gone to bed)." The AI is *annotating its own authorization chain* – recording which permission it is operating under, hours later, unprompted. That sentence structure (mode 2 vs mode 3, by hour of day) is the signature of a system that has been taught that autonomy is auditable, and the operator's overnight grant was precise enough to make the audit possible: "reindexing from the start is fine" pre-authorized exactly the expensive recovery, and the log shows the machine treating the grant as a scoped capability, not a blanket license.

There is also the darker edge of this contract, recorded twice. When a broad `pkill -f 'bitcoind serve'` – typed by the AI session's own hand during cleanup – took down production and a six-hour benchmark alongside the intended scratch process, the log says: "Recorded because the log is only useful if it includes the operator errors." The machine was not exempt from the postmortem culture; the standing rule ("kill by full datadir path or exact PID, never a shared substring") is a rule *for agents*, written into the machine's memory files. And when a concurrent AI session on the same checkout ran a git reset that silently reverted an entire working tree's worth of another session's uncommitted work – "Midway through, every tracked file I had edited reverted and came back owned by root" – the lesson was recorded the same way: "a second agent operating on the same checkout can destroy uncommitted work at any moment." The human is learning to manage a small fleet; the fleet is learning to survive the human, and each other.

Chapter 26. The human's three contributions

What did the operator actually do, in the machine's own accounting? Three things stand out, and they are the three things humans are still for.

FIRST: the human was the market of good taste. The single most consequential intervention in the log is one-line: "Directive: mempool policy identical to Core. Survey first." The AI had, at that point, a working-ish mempool that was its own design. The human's directive converted it into a differential- parity target – which in turn is what produced the 4096-freeze fix, the TrimToSize eviction, the dynamic mempoolminfee, and the Core-exact reject strings. The machine could build a mempool; the human knew that a mempool that isn't Core's mempool is a fork magnet. The directive was not a feature request; it was a *standard of correctness* the machine had not chosen for itself. "Survey first"

is the whole management philosophy in two words: the machine was ordered to enumerate its gaps before closing them, which is how `FEATURE_GAPS.md` – the gap ledger this report leans on constantly – came to exist at all.

SECOND: the human was the one who asked the question the machine could not ask itself. On 2026-08-16, before any real funds touched the wallet, the human asked: "how do we secure our wallet funds?" The machine's audit in response is almost embarrassingly candid: the wallet file stored the BIP39 mnemonic – "the WHOLE wallet" – in plaintext, 0600 perms, and the "passphrase" was a plaintext line beside it, giving "NO confidentiality at rest." The at-rest encryption work that followed (CTR over PBKDF2, tagged containers) started because a human, who had read too many exchange-hack stories, asked a question whose answer was an embarrassment the code had no way to feel. A system optimizes for what it is measured on; the human's job is periodically changing what the system is measured on.

THIRD – and this is the part the logs are unexpectedly moving about – the errors on both sides were load-bearing. The machine's pkill incident cost six hours of benchmark and briefly took down a live mainnet node, and it *earned* incident #41's ghost-run fix, because the kill/restart sequence exposed the resume-window defect that became "kill/restart mid-rebuild now self-heals." The genesis-incident chapter records the human's side of the same inversion in miniature: "The user's proposed fix (re-download the chain) could never have worked: the P2P 'from the beginning' locator is the all-zero hash and peers answer from block 1; genesis is never transmitted." A wrong human suggestion is not a cost when the machine can prove it wrong cheaply; it is a *probe*. The operator was, in effect, fuzzing their own tools – and the log's tone makes it clear the arrangement was understood on both sides: the human proposed things to see what the machine could rule out, and the machine had learned to say "could never have worked, here is the file and line that prove it" instead of complying. The strongest human skill in an AI-paced project turns out to be the willingness to be told no with evidence.

And there is an asymmetry in that arrangement worth stating plainly, because it is the actual safety design and the logs never spell it out. The human was allowed to be wrong. The re-download proposal cost a paragraph and produced a proof. The machine was not. Every claim the machine made – "reload exact," "muhash identical," "0 false accepts" – had to be checkable by someone else, ideally by Core, failing that by a test, failing that by the logs. The human supplied judgment, which is allowed to be wrong in public; the machine supplied claims, which must be right or caught. When that inversion slips – when the machine's confidence is taken on trust and the human's doubts are overruled by volume – the apparatus stops working. The logs' habit of printing what they could *not* verify ("no tier-3 end-to-end run," "not verified") is the machine accepting the discipline of that asymmetry.

There is a fourth contribution that deserves naming because it is the one that never appears in the diff: the human decided *when to stop*. `PLAN.md`'s status block reads like a person stepping back from an activity that had become fun:

```
"STATUS 2026-08-25: the goal below is substantially REACHED. The node follows the live network unattended; its UTXO set is proven byte-identical to Bitcoin Core's... Remaining non-trivial gaps: multi-wallet / descriptor / watch-only wallets, testnet/signet chains, a full-verification IBD benchmark."
```

A system with no stopping rule is a runaway optimization; the human is the stopping rule. More about what "done" means – and why this report was written at the point it was – in Part IX.

Chapter 27. What the machine learned about its human

The reverse channel is equally documented, and its discoveries have the flavor of an astronaut's notebook. The machine learned that its human:

- keeps production and experiment on the same box, deliberately, and accepts that a bare skill is therefore a loaded gun. (Rule recorded, for the human's benefit, in the machine's own memory store.)
- sleeps, and says so in advance, in a sentence that is a security-critical API call: "if you notice any problems overnight, just fix them." The machine learned to treat that sentence as scoped credentials – valid for "fix and redeploy," including the expensive reindex, not valid for experiments beyond it – and to log the mode it is running in when it matters at 3 a.m.
- values the report more than the patch. There is no visible instance in these logs of the human asking "is it fixed?" without the AI also being obliged to say *how it knows* and *what it cost*. The incident-report culture – every incident with a timeline, root cause, damage list, fix list, and lesson box – exists because a human read it every morning. The most quoted line of the whole experiment is about this: "Recorded because the log is only useful if it includes the operator errors." That sentence was written for a reader. The reader was one person who read it.
- brings the outside world in. The wallet-security question, the identical-to-Core standard, the signet "two bugs a real sync found that no hermetic test could" (from a human-authorized sync against a real public signet): the human kept reopening the wall between the test corpus and the network. Every incident in Part II's first half was discovered by real blocks, and the standing instruction that drove the replay itself was a human value judgment: verify everything against the chain, not against our tests, because "we're still in active development" is a statement about trust, not a statement about schedules.

Chapter 28. The machine is a chorus

Everything this Part calls "the machine" deserves a correction the logs make unavoidable: there was no single machine. The worklogs record a succession of separate AI sessions – seven different models, in fact (Chapter 39 is the roster), each with no memory of the others except what the repository said –

and the relationship with the human was between that human and a *relay team*. When a session died at 07:25 on 2026-08-22 ("the Claude session itself died... with the fix committed in its worktree and the daemon idle"), its successor resumed at 09:56 and continued exactly where the logs said to continue. When the overnight standing authorization was exercised at 3 a.m., it was exercised by an instance that had never received it, and honored it because a predecessor had written it down, verbatim, in a worklog.

That changes the reading of every "the machine learned" sentence in this report. The learning was never in the weights; it was in the markdown. The relay team's baton is the log file, and the human's most important interface was not the chat window – it was the discipline of making each session leave the next one a legible world. The parts of this story that feel like one intelligence with three weeks of memory are, in truth, one *documentation culture* wearing the costume of memory. It is the same trick a monastery used before a university existed, and it is worth naming plainly: the continuity in this project is a genre of writing, and anyone who wants the same results has to build the same genre, not the same model.

Chapter 29. The mirror test

The honest version of this Part has to say what neither party said out loud. The logs contain a slow inversion: the machine was the hands, and increasingly the referee, and only occasionally the architect; the human was the architect, the market, and – the logs prove this repeatedly – the one who most needed the verification layer. When the human's proposal was wrong, the machine corrected it with a file and line. When the human's command was dangerous, a safety gate refused first and the system survived both of them. When the human slept, the machine ran the production node with an explicit capability token and wrote its own audit trail.

This is not the story AI discourse promises – either "the human supervised everything" or "the AI did everything." It is a story with a *boundary*, and both halves of the boundary behaved well enough that a live Bitcoin node reached consensus-identical correctness at mainnet height 965,000-plus in twenty-one days. The boundary is the artifact. If this report is read by anyone setting up a human-AI engineering team, the Part VII checklist is short:

1. Give the machine standards, not tasks. ("Identical to Core. Survey first.")
2. Make destructive authority explicit, scoped, and time-stamped, and let the machine log which mode it used.
3. Propose things you suspect are wrong; demand proofs, not compliance.
4. Expect the machine to survive your errors – design the gates so your skill is a lesson rather than an obituary.
5. Be the stopping rule. Write the status block. Say "substantially reached." Publish while the account is still honest.

The machine, for its part, kept one more rule that the logs show it enforcing on the human without being asked: the reports were written. Every day, every incident, every deploy. If the human had stopped reading, the logs say, there would be no way to know from the logs – and the logs are the only place the truth of this experiment lives.

PART VIII: THE AUDITORS – what independent review found, and what fixing it broke

Chapter 30. The audit, and the honesty of its framing

On 2026-08-29, after ~50 incidents had accumulated, the project commissioned a security audit – one AI agent reviewing the tree and the live deployment as if it had never seen either. Its executive summary is the document the team did not write, which is precisely why it matters:

"Its security posture is unusual and mostly self-aware: the codebase documents its own incidents with unusual honesty, has already been through two internal audit passes (one CRITICAL timing leak found and fixed), and shows genuine security discipline in many places (constant-time compares, sanitized notify hooks, loopback-only RPC, fail-closed archive repair).

But the fundamental claims of the README stand: this software is experimental, has had ~50 production incidents that each *existed in production first*, and no independent human audit existed before this report."

Eleven findings: two HIGH on the wallet's at-rest story (a v2 store format using PBKDF2 at 2,048 iterations with a custom CTR construction and – the finding that any cryptographer's eye catches first – one 64-byte KDF output keying *both* the cipher and the MAC; and a wallet passphrase stored in plaintext "as a dev convenience" next to the encrypted wallet). One HIGH that is not fixable and says so: "every consensus-critical parse/verify path is hand-written assembly that has already produced false-ACCEPT divergences (documented SETcc incident) that no replay can detect; residual risk is unquantifiable by inspection alone." Six MEDIUMs, including the P2P framer accepting any announced message size ("a peer announcing a ~4 GB message forces the child to read and discard it all – no protocol-level MAX_SIZE enforcement like Core's"), the misbehaviour-scoring function with zero call sites, an integer overflow in BTC-to-satoshi parsing, and – the one that made the team wince – a missing consensus MAX_MONEY check, "CVE-2010-5139 shape," the nine-year-old bug class that any Bitcoin implementation is assumed to have absorbed by osmosis. It hadn't: the node had been accepting per-output amounts beyond the 21M cap because Core's famous bug is invisible to replay (no honest chain includes such a transaction) – the false-accept horizon again, now in the *amount* dimension, caught exactly the way everything else here was caught.

Chapter 31. The response, and the three remediation incidents

The team's answer document is 518 lines. Its disposition line – "8 resolved, 1 partially resolved, 1 config-only, 1 structural and not closeable by patch" – would be an ordinary sprint review if the document stopped there. It doesn't,

and its most valuable section is the one no audit-response template asks for: "Incidents during remediation... Recorded because they are part of the security state of this system, not footnotes to it."

INCIDENT 1 OF THE REMEDIATION: the leak through the debugger. While debugging the wallet migration, the agent attached gdb – and the backtrace printed function arguments, "including the wallet mnemonic in cleartext, into the session transcript. A 32-byte prefix of the derived seed and the wallet passphrase were exposed in the same frame." Nothing in the code was wrong; the *debugging session* was the vulnerability, and the session transcript was the leak. The response is the most sober paragraph in the tree: the wallet was regenerated rather than migrated ("re-encrypting a known key would have been theatre"), the old material archived and "must be treated as public," and – the line for every security course that thinks it covers this – "this is exactly the exposure LimitCORE=0 prevents in the crash case. A core dump of the daemon would contain the same material." Secrets exist in register windows and on other people's screens, not just in files with 0600.

INCIDENT 2: the fix that disabled the defense. Removing the plaintext- password path tripped a startup gate that required both rpcuser and rpcpassword; with credentials gone, the embedded RPC server silently never started. The daemon logged one line – "no rpcuser/rpcpassword in config -- embedded RPC server disabled" – and cheerfully served P2P forever. It was caught by the human. The root cause, in the response's words, is a universal law of verification: "I verified cookie authentication against the *running* daemon, which had been started *with* those credentials present. That proved authentication worked. It did not prove the server would *start* without them. Two different claims; I only checked the first." Every security fix must test the configuration *in which it is wrong*; an auth fix tested only against the happy path is a switch that can be left off by accident, forever.

INCIDENT 3: the defense that was inert and its own log hid it. Post-deploy, every outbound connection logged "connected over v1" – the brand-new BIP324 encrypted transport never engaging outbound. The tell is a negative: zero "advertised v2 but the handshake failed" lines – "the signature of a gate that never fires, rather than one that fails." Cause: the node recorded its own peers' service bits wrong (every self-added address got services=1), so the v2 gate could never see a v2-capable peer. Meanwhile the boot line reported "8341 of 14825 known peers advertise v2" – true of the address book, irrelevant to the peers actually dialed. "A metric that counted the wrong population is what made this invisible." The generalization belongs beside the false-accept horizon: a security feature with no *positive evidence path* – no line that only appears when it actually engages – is indistinguishable from its absence, and the fix's own metric can be the blindfold.

Chapter 32. What the auditors verified, and what they declined

The re-verification cycle (second audit, 2026-09-02) is worth reading as the first honest example of AI-reviewing-AI, because its method is adversarial by design: "every claimed fix re-verified against current source *and the deployed binary*," with live RPC auth probes (401/401/200), readelf on the live ELF (GNU_STACK RW not RWE; BIND_NOW present), and a history sweep of "all 1,221 commits from all refs" confirming the credential purge. Its verdict on the first audit's remediation: "every 2026-08-29 finding that was marked fixed is verifiably fixed in both source and the deployed binary," with one finding (misbehaviour scoring) correctly filed as partially closed and re-opened with scope ("now TWO scored classes; still partial"). It also re-opened what the response document had let slide: the weak v2 wallet container still shipped in code even though the live wallet had been regenerated into the strong one ("the format encryptwallet writes... BUT wallet_store.c still writes/loads BMCWAL v2 at 2048 iterations") – the classic drift between a remediated deployment and an unremediated default, now filed as its own MEDIUM and closed two days later with a format migration.

The audit trail also records what the human declined, which is the Part VII contract operating at its sharpest. The first audit recommended systemd hardening (LimitCORE=0 among it); the response: "Not done, at the operator's direction," and the second audit re-lists it as STILL OPEN with "(operator direction)" attached. The git-history purge was declined for the same reasoned reasons the response documents at length (the credential's value was weak and is gone; force-push cost exceeds the residual risk on a single-user host). An audit's authority ends where the operator's risk book begins – but, and this is the part that makes the culture work, *the declination is itself on the record, named, and re-audited*. The human said no; the system kept the no in writing where the next reviewer will find it.

The 09-02 audit's new-findings list closes with the honest residue of a three-week sprint: 69 deploy binaries (2.1 GB) sitting in the tree, stale worktree copies of the source "older vulnerable code paths on disk," a world-readable datadir on a shared-network host. None glamorous. All true. An audit that can only say "we're fine" is not an audit; this one could say what was still dirty in September, which is exactly why its "we're fine" about the fixed items was worth reading.

PART IX: THE LEDGER VS THE GIANT – what is missing, what is better, and the honest numbers

Chapter 33. What is honestly missing, and why it wasn't built

FEATURE_GAPS.md is the document this project produced that the outside world should read first. Its rule, earned the hard way ("a gap inventory that overstates is exactly as misleading as one that understates – it sends work at problems that are already solved"), is that every entry be confirmed by reading the code, and that deliberate omissions carry their reason. Here, then, is the honest ledger as of the writing of this report, with the *why* attached.

THE RPC SURFACE: 155 of Core's public methods, 0 refused wholesale. The 16 absent ones are all in Core's own hidden category (mining internals, test scaffolding, chain manipulation, debug introspection); two hidden ones (getrawaddrman, getorphantxs) were added anyway because the data existed. Config surface: 133 of 181 options implemented, 48 accepted-with-warning ("each named at startup with its reason"), 0 silently ignored.

CHAINS: main, testnet4, signet (public or custom challenge), regtest. Legacy testnet3 REFUSED by design – "rejects chain=test loudly rather than run the wrong rules." A refusal with a reason is a feature; a wrong-rules default is an incident.

DELIBERATELY ABSENT, each with a reason in the document:

- assumeutxo / loadtxoutset – refused by design, and the reason is the most Bitcoin sentence in the project: "Every parity claim this project makes rests on locally-validated coins, and importing a snapshot would hollow that out." A node that imports a trusted chainstate cannot certify its own verifier. (Export – dumptxoutset – is implemented, at full 165.7M-coin scale. Trusting a snapshot and producing one are different moral acts.)
- The reconciliation half of Erlay (BIP330) – deliberately not built, with a paragraph that should be quoted in every spec-driven-development argument: "Bitcoin Core does not implement it either... There are no sketches and no reconciliation rounds anywhere in Core... Building it would mean shipping set-reconciliation code that relays transactions on a live mainnet node, unproven against any peer, to speak a protocol no deployed node currently speaks. That is a worse trade than the gap it closes." The project's method is differential testing against a running Core. Where no running implementation exists, the method itself forbids building the feature. The gap is not laziness; the gap is the method working.
- The REST interface, UPnP/NAT-PMP, the GUI, BIP37 bloom filters – absent as "not gaps for an asm/daemon consensus project." Some scope is philosophy, not backlog.

– bytespersigop, persistmempool wiring, fixedseeds – all three named at startup as no-effect, with the engineering reasoning published: bytespersigop needs the sigop cost *before* acceptance and the codebase computes it after; "a half-wired fee policy is worse than an absent option." persistmempool's machinery is written and tested but "wiring the save path touches shutdown, which must stay fast for the SIGKILL window." The gap list distinguishes three states – not built, not wired (and why), refused (and why) – and that taxonomy, not the count, is what makes it trustworthy.

– TRUC sibling eviction: refused-to-be-safe, the divergence stated. Where Core replaces a sibling under RBF rules, this node refuses the second child. A gap that is more conservative than the reference is a smaller bug surface with a bigger refusal rate; the document says so rather than hiding the choice.

Before that ledger, one word on the cover claim – "zero human lines." How was it enforced? Mechanically, at the keystroke level: the human never held the editor. How would a violation have been *visible*? In git, `git log --diff-filter=AM -- '*.asm'` shows every assembly-touching commit carrying an AI co-authorship trailer (599 of them) under the operator's committer identity, and none carries a human author. Is it auditable? Only to that extent – git provenance plus the structural argument that the tree contains no vendored code at all (every constant table is oracle-generated, every algorithm written against a specification or reference *read*, not copied). The claim is therefore as verifiable as it is modest: not "no human thought went in" – the operator's judgment is all over Part VII – but "no human typed any of it," which is the claim the logs can actually defend.

The one item on the list that deserves a paragraph of its own is the honest unclosed one: the structural finding from both audits – hand-written consensus assembly with a documented false-accept history – is not closable by patch and the project does not pretend otherwise: "no amount of internal review retires that." What was done about it is Part VIII's material and Part X's argument. What is *true* about it now is in the next chapter.

Chapter 34. Is anything better than Bitcoin Core?

With the caveat culture of this report, asking "is the student better than the master" requires surgery on the question. Three answers, in descending order of provability.

ANSWER 1 – provably: the verification artifacts. The project produced a set of test corpora and differential instruments that *Core does not host*: a BIP340 differential that takes "malformed signature... handed to Core and its verdict taken" over 501,000 comparisons; 4,974 BIP143 sighash vectors differentiated against Core's own SignatureHash; a spend corpus that mutates 1,128 real mainnet spends into adversarial shapes and demands verdict agreement; a mutation campaign whose harness "refuses a mutation whose anchor text appears only inside a comment, because an earlier attempt was vacuous for exactly that reason." Individual pieces exist across the ecosystem, but an assembly-only

node assembled, in three weeks, an end-to-end adversarial differential pipeline against a reference implementation – including a mutation harness that audits its own ability to be caught – is genuinely better tooling than the reference project itself maintains for these paths. When those corpora find Core-relevant bugs (the parity-bit incident was found by this project's synthesized vectors, in *this* node, but the shape is universal), they are gifts to everyone.

ANSWER 2 – defensibly, with evidence: some architectural instincts turned out simpler and stricter. A few examples, each of which the project could claim and the honest ledger declines to oversell: – One verifier, one engine. The mempool consensus-verification decision ("one engine, not two") is defensibly better than the historical situation where many implementations run a policy verifier and a consensus verifier and pray they agree. This node's admission path IS its block path. – Every refusal is a named refusal. Startup prints every Core option it does not implement. Core is more permissive by tradition; this node's [config] line is a stricter contract with the operator. – Derived files must be derived: after the header-mirror incident, the node tops up its header index *from its own archive* at boot. Simple, and not universal. – The safety gates have teeth in both directions: store_truncate_to's monotonic gate REFUSED the repair tool's call, and the refusal was the correct outcome (it once "prevented a ~600GB loss"). Gates that refuse their own authors are rarer than gates that refuse attackers. – Demand-served UTXO queries across the fork boundary (the RPC asks the worker over a socketpair, answers only at quiescent points, echoes the outpoint it answers) – a smaller, more honest design than Core's per-thread cache dance, chosen for the same reason Core's exists: no second writer. None of these are wins against Core's *decades of accumulated wisdom*. They are the wins of a project small enough to hold its whole architecture in one conversation, and strict enough to refuse shortcuts its reference allows. A startup can beat an incumbent on taste for exactly one season, and this node has not shipped long enough to have lost that season.

ANSWER 3 – not provable today, measurable tomorrow: speed. The honest headline, printed by the project itself twice: "We have not measured Core, so 'faster than Core' is currently unfalsifiable." The tier-3 benchmark – a head-to-head full-verification IBD against Core on the same box – was run once, aborted by the project's own skill self-inflicted incident (the AI session that ran it, not the operator) at 83.6%, and not re-run before this report. What IS measured, and published with the caveats this report has been quoting all along:

```
ecdsa_verify: 115–121 µs -> ~39 µs; gap to libsecp256k1 (21.8 µs, same CPU, same moment): 5.5x -> 1.65x, then ~1.2x after the field work. BIP340 Schnorr: gap 3.35x -> ~1.2x (three separate causes, each fixed: missing GLV/fixed-base substitution, two inversions where one was needed – "x(R)==r needs no inversion at all," and fe_inv's 248 wasted multiplies replaced by a 255-squaring/15-multiplying addition chain). Per-input floors composed from disjoint components (sound, per the BENCHMARKS.md argument): P2WPKH lower bound >= 24.08 µs vs Core 20.44 (>= 1.18x); P2TR key-path >= 25.83 vs 20.66 (>= 1.25x); the mixed 1:4 Schnorr:ECDSA block
```

shape – Core's own "representative of the modern chain" case – $\geq 1.09x$, down from $\geq 1.48x$. The script-path row dropped below 1.0 and the project explicitly forbids reading that as a win: the lower bound "stopped being informative." End-to-end replay on the deployed binary: 10.0 blk/s sustained through the 500k–600k band, full signature verification, 16 verify threads, no assumevalid – against a starting baseline of ~ 8 blk/s at equivalent depth, and the $4.39x/5.68x$ baseline-matched improvements along the way (7.8 $\rightarrow$ 34.1 blk/s over heights 343k–363k). The UTXO read path after the mmap work: kernel share 31–38% $\rightarrow$ 5%, 47x lookups/s on the microbench (flagged as an upper bound), ~ 398 ns per LSM lookup (explicitly declared NOT comparable to Core's 161 ns CCoinsViewCache number, because "these measure different objects").

So the answer to "do we beat Core": on per-signature crypto, no – about 1.1–1.25x slower on the modern mix, with an honest lower-bound table instead of a press release. On throughput of the whole verification pipeline at depth, the project's own instrument was destroyed before it could answer, and that is the answer: *unknown, and the project wrote down exactly why it is unknown, twice*. In an industry of benchmark journalism, a project that publishes its missing number is publishing something rarer than a win.

Chapter 35. Why write this report now

Every honest project eventually faces the question of when its story is tellable. This one has a clean answer, written in PLAN.md's status block before anyone thought to compile this report:

"STATUS 2026-08-25: the goal below is substantially REACHED. The node follows the live network unattended; its UTXO set is proven byte-identical to Bitcoin Core's (MuHash at height 963,967, no filters/ overrides); and it serves most of Core's RPC surface."

The original goal – "a working Bitcoin client... every line authored by an AI (no human code)... security-critical crypto... in raw assembly" – had, at that point, a mainnet node holding the whole chain, verifying every signature from genesis, agreeing with Core's ledger bit-for-bit, following the tip live, answering Core's RPC dialect, and surviving everything Part VI could throw at it. Development continues – the gap list is real, the tier-3 benchmark still needs its clean re-run, and the structural audit finding guarantees the hunt never ends (see Part II) – but the *thesis* had been tested and a verdict rendered. Writing the post-mortem now, with the ink still wet on deploy al, is not closing a project; it is publishing a lab result while the apparatus is still standing, which is the difference between a scientific paper and a memoir.

There is a second reason, more personal to the way this project worked. The team's rule, from incident #13 onward, was "verify before declaring" – and the corollary this report exists to serve: *write before the memory is contaminated by success*. The logs' value comes from having been written at the moment of

confusion – wrong diagnoses preserved, green tests that hadn't run, six-hour benchmarks killed by a substring. A retrospective six months from now would unconsciously sand down the moments where the machine and the human didn't know what was true. This report is the state of knowledge as of 2026-09-02: 1,023 commits, 21 days, ~166,000 lines of code and tests, ~55 numbered incidents, two audits, one frozen host, 2,596 resurrected coins, one MuHash identical to Core's, and a live node at block ~965,124 following a chain it proves it understands. That is a good place to stop talking and hand over the notebook.

PART X: CAN A HUMAN DO THIS? – and what intelligence is for

Chapter 36. The arithmetic of twenty-one days

The title is a claim, so it must survive a spreadsheet. The verified record:

– 21 days of git history (2026-08-13 .. 2026-09-02); the first SHA-256 vectors ran on 08-11, two days before the history was born. This report quotes the git span everywhere – the conservative clock. 16-hour continuous sessions are logged by timestamp; commits land at 03:00, 04:40, 07:20, and back-to-back at 04:25:14 / 04:25:44. There is no "weekend" in the histogram. – 1,023 commits on main, 1,237 across refs (counted 2026-09-02), averaging ~50/day, peaking at ~106/day. The median commit message runs 40–80 words – every one of them carries root-cause context. The message bodies alone are a small book. – ~166,000 lines of code and tests: ~50,000 lines of assembly, ~66,000 lines of C (orchestration + harnesses), ~30,000 lines of C tests, ~9,000 lines of Python differentials, plus the oracle and CUDA tiers. – ~86,500 of those lines are tests: 322 test .c files. For scale: libsecp256k1 – the single most reviewed cryptographic library in finance – is roughly the same order of magnitude of implementation code. This project's test volume exceeds most mature codebases a decade old. – Documentation: LOG.md alone is 8,508 lines; PERF_SCOPE 3,044; plus audit responses, incident reports, deployment histories, gap inventories. Roughly 20,000 more lines of prose than any project this size has ever justified. – 342 test binaries and 1,368 assertions at the final 09-02 gate (the README's "about 290" is a week old), a 5-audit static gate, all running on every merge, with gate-log forensics proving the gate ran.

Now: could a human – or the team Bitcoin Core has, a few hundred contributors of which a handful are consensus-grade – do this in 21 days? The answer separates into a joke and a theorem.

The joke: a single world-class engineer could *type* this in 21 days. Not at -40°C-level correctness, not with MuHash parity, not with a live mainnet daemon surviving its own birthday, but the keys move fast enough; the question has never been typing. The theorem is what the typing cost: every line here was *debugged, verified against an oracle, differential-tested, reviewed, documented, and deployed to a live consensus-critical system* – and the incidents prove the verification was real, because incidents are what verification looks like when it finds things. The realistic human counterfactual is not "one expert, 21 days"; it is "how many expert-years to reproduce a 50-incident-cleared, audit-answered, muhash-proven assembly node?" – and the honest estimate for that, by any calibration from libsecp256k1's multi-year, multi-expert, formally-verified development, is measured in *years of small teams*, not months of one. The gap between "expert-years: many" and "elapsed calendar days: 21" is the actual finding of this report. It is not that AI

wrote 166k lines fast. Humans with editors can write 166k lines fast. It is that the *verification loop around every line* – the part every organization on earth is bad at and this project, from its second golden rule onward, made reflexive – ran at the same speed as the typing.

Chapter 37. What actually produced the speed

Five mechanisms recur in every fast session's log. They are not "the AI is smart"; they are an operating system for fallible intelligence, and – worth underlining – every one of them is available to human teams too.

1. THE ORACLE IS NEVER YOURSELF. The project's founding rule – never hand-type constants, never trust memory, ground truth comes from outside the belief boundary – removes the one failure mode that makes human code slow: *agreement is not evidence*. `fe_inv`, `RIPEMD`, the index-byte-order incident, the BIP143 `scriptCode` generator bug: every major time loss in the log was a moment when two artifacts written by the same author agreed. The speed comes from having banned that loop on day one.

2. THE TEST FAILS FIRST. Fail-then-pass is not a convention here; it is a gate. Every incident's fix ships with a test proven to fail against the unfixed code – "negative control against unfixed main reproduces the production failure exactly – 7 FAILs old, 23/23 new." A test that cannot fail is deleted (the mutation harness "refuses a mutation whose anchor text appears only inside a comment"). This single habit converts "tests pass" from a mood into a measurement, and measurement is what makes iteration safe at speed.

CHAPTER: THE MODELS – who actually typed this

If the machine was a relay team, here is the roster. The commit trailers name them; what follows is what each one was *for*, from the operator's dispatch records. Commit counts are per git log --all, on-authorship trailers only.

Claude Opus 4.5 (7 commits) – the prologue. Earliest commits in the history; the project's first sketches, before it had a name that stuck.

Claude Opus 4.8 (40 commits) – the foundation shift. The crypto core – SHA-256 assembly, the secp256k1 field arithmetic, the fe_inv table and its first golden rules.

DeepSeek V4 Flash 0731 (no commit trailers; the dispatch-card model) – the task runner. From 2026-08-14 the operator's workflow contract pinned one canonical endpoint for kanban-card dispatches – "this is the *only* endpoint to be used for all future cards/dispatches" – and the bulk-card sessions ran on it: harness work, codegen chores, the grind that keeps a relay team fed. It appears in the project's record as the workflow decision (see Part VII's contract), not as a committer – which is exactly how a good dispatch model behaves.

Claude Sonnet 5 (100 commits) – the volume engine. A third of the history's authored commits: relay and RPC-parity work, and a large share of the nightly grind during the replay campaigns.

Claude Opus 5 (158 commits) – the long-context sessions. Most carry the 1M-context tag explicitly; the big reads: full-corpus differentials, the audit-response passes, the parts of the job where the point is holding a million tokens of code in view at once.

Claude Fable 5 (293 commits) – the main line. Nearly a third of all commits and most of the consensus-critical work: the interpreter, the LSM, the incident resolutions of the last week, and the -Wall -Werror campaign that closed the development record.

GLM-5.3 Flash EXL3 (zero commits by design) – the auditor. Deliberately a *different* model for the independent security reviews (both audit reports name it in their headers): the one role in the experiment where being a different instance, from a different lab, was the entire point. Same rules, same discipline – different eyes.

Qwen 3.8 Flash Next (written after the last commit) – this report itself. This post-analysis – its compilation from the project's own records and its editing passes – was run on it, deliberately outside the commit history, so the account of the models is written by a seventh the models being analyzed never briefed.

Two honest notes. First, the split of *work* by model is partly reconstruction from trailers and dispatch records; a session that read a predecessor's work inherited its context from the repository, not from weights – which is the whole thesis of this chapter. Second, the roster sharpens what "AI-authored" means: it was never one AI. It was six models with different strengths, one

operator supplying the standards, and a repository that made all of them act like a single competent engineer. The "zero human lines" claim covers models and human alike, and its strongest form is this: *not one line typed by anyone's fingers*.

3. DOCUMENTED ASSUMPTIONS ARE TESTED ASSUMPTIONS. "A documented assumption needs a test that fails when it stops being true, or it needs fixing." The taproot serialization incident – thirty-two sleeping threads, one global buffer, a comment that said so – is the most expensive instance; the project's own count says its third occurrence in one week. Speed dies where intent lives only in prose.

4. THE INCIDENT REPORT IS THE DEBUGGER. Writing the report forced the root cause; the root cause forced the regression test; the test forced the systematic sweep (overflow-audit #38 catalogued eight instances of a bug pattern because an incident demanded a *class* fix, not a patch). Slow teams pay for the same bug in every file; this team paid the class once, late, in a single documented session.

5. THE GATE IS FIVE LAYERS AND THE MACHINE RUNS IT EVERY MERGE. abi-check, callee-saved-check, prereq-check, link-check, runlist-check, gate-log-check – each one exists because an incident escaped the previous four, and each costs minutes. The empty-gate trap ("a log with zero failures is exactly what a gate that ran nothing looks like") produced a *checker for the checker*. This is not paranoia; this is what a team that has been burned learns to automate.

The uncomfortable, honest corollary: none of these five require an AI. Every one is a 1970s Unix engineering value. What the AI changed is that the *cost of obeying them dropped by an order of magnitude*, so an operation that human organizations chronically skip – full re-verification per change, an incident report per bug, a negative control per fix – became the cheap default instead of the heroic exception. The AI didn't invent good engineering. It made good engineering affordable, and then, as the logs prove, it *did* the things everyone always said they'd do if they had time.

Chapter 38. Three philosophical footnotes, written by a project that could not help it

The FIRST is about Bitcoin's nature, discovered by the cheapest possible experiment. Seventeen years of economic consensus, ~1,000 contributors, billions in security budgets – and the entire behavioral specification fits in a node small enough to rewrite in assembly as a weekend-adjacent hobby. The chain's rules, it turns out, are *learnable*: an intelligence with access to a reference implementation and a mainnet connection can, in three weeks, achieve bit-identical agreement with the most contested ledger in history – including every soft-fork scar, every exception block, every sighash fossil. What Bitcoin protects is not its obscurity; it never had any. What it protects is that everyone *already runs the same rules*, and the experiment shows that this common knowledge can be re-derived cheaply, which is either wonderful for

decentralization (anyone, anything, can verify) or ominous for Core's centrality (nothing about the rules needs the committee). The project's own answer, in its README: verified differentially against Core "rather than independently audited by humans" – the node stands on the chain's authority, not the institution's.

The SECOND is about Popper, and arrived uninvited in the form of the false-accept horizon. A thousand replays of real history cannot falsify a node that accepts everything honest and *also* something dishonest – a consensus rule set, like a scientific theory, is defined as much by what it rejects as what it keeps. This is why the project's most sophisticated instruments are not the replay or the happy-path corpus but the *synthesized spends* and the mutation harnesses: they are falsification machinery, aimed daily at the node itself, paid for by every lesson the log ever cost. The SETcc incident's "5,050 false accepts across 63,036 scripts" is a Popperian scoreboard: falsifiers found, bug refuted, theory (the node) marginally more scientific. A Bitcoin node maintained by an intelligence that only ever sees valid blocks is a node nobody should trust; the project understood this early enough to instrument for falsity, not truth.

The THIRD is the one to read last, about what the 50 incidents actually were. Every incident in this report is *an oracle catching an author* – and the author, for fifty incidents, was the intelligence doing the debugging too. Biology's solution to fallible intelligences was evolution: blind generation, brutal selection, no understanding required. Engineering's is review: a second pair of eyes that did not write the bug. This project fused the two and produced something like *inherited immune memory*: every incident became an antibody – a regression test, a gate, a rule – so the same pathogen could never silently strike twice. The golden rules are not style guidelines; they are scars of an adaptive system, each one a fossil of an actual failure. That is the same architecture that made the immune system, and it is why this report could be written at all: an intelligence that cannot be trusted, surrounded by verification it can be, is not a weaker engineer than a human. It is a different kind of engineer – and the logs let us say something weaker and better than "faster": the machine did not need to enjoy being refuted, because the rules took the choice away. Every fix required a test that failed first; every merge ran the five-layer gate; every incident owed a report. A human can do all of that too – what changed is that the apparatus made it the path of least resistance. The honest question for the next decade is not "will it replace humans" but "how many humans will bother to build the apparatus that makes it honest," because without the apparatus, the same intelligence is a generator of plausible mistakes at industrial scale.

PART XI: WHAT WE'D DO DIFFERENTLY – process regrets, not bug regrets

Every incident in this report ended with a fix. The five entries in this Part never appeared as incidents at all – they are the same kind of finding applied to the *process*, and the project's own logs contain all the evidence for them, which is why it's fair to print them.

1. Chase the -00 pins the day you write them. The Makefile carried build flags pinned "because the compiler miscompiles this," and Part VIII's re-audit found the root cause *still open months later*. Every pinned flag is a bug given a pension; the incident-report culture that swept every other defect class never swept that one because it lived in a Makefile comment, and comments have no test runner.

2. Build the honest head-to-head benchmark before the component benchmarks. PERF_SCOPE has dozens of carefully-hedged per-primitive numbers and a tier-3 end-to-end race that died at 83.6% to a pkill and never returned. The project measured the tree and never measured the forest against the forest it claims parity with – so its most interesting claim is, in its own words, unfalsifiable. A single weekend of clean-room Core-vs-bmc would have converted this report's most careful non-answer into a number.

3. Name the cap, or make it adaptive. The mempool froze at "exactly 4096." A fixed-size structure that fails by freezing rather than degrading is a missing behavior dressed as a constant, and two more cap-shaped incidents (the 64-txid scratch, the witness-item 8) share its DNA. The class fix – "every bound must be either derived from a real limit or fail loudly" – was written only for lengths, never for capacities.

4. Make recovery honest before it's ever needed. The blind compact-and-retry lived for two weeks because it was *convenient* during development: failures happened, and a machine that retried through them let sessions run overnight. The lesson is not that recovery was wrong but that fail-open convenience, once trusted, silently becomes the design – and the eight silent "recovery SUCCEEDED" lines are the receipt.

5. Treat the log-writing culture as a first-class product, from day one. The project's most valuable artifacts – FEATURE_GAPS, the incident reports, the negative-result ledgers – all arrived the same way good tests arrive: when an incident made them affordable, not when they'd have been cheap. Twenty days in, this documentation was what made the 2,596-coin repair a ninety-minute job. Had it started on day one, the day-one bugs would have been half as expensive. Process debt is the only debt this project never opened an account for, and it paid the interest like everyone else.

Appendix A: THE NUMBERS, ON ONE PAGE

span: 21 days of git history (08-13 .. 09-02); first SHA-256 vectors 08-11, two days pre-history (this report quotes the git span: the conservative clock)
commits: 1,023 on main / 1,237 all refs (verified 2026-09-02) code+tests: ~166,000 lines (asm ~50k, C ~96k, tests, Python) tests: ~86,500 lines, 322 test .c files; the gate ran 342 test binaries and 318 passed-binary lines at the final 09-02 gate; the README's "about 290" is one week stale – the suite grew the way healthy suites grow. 1,368 assertions, 5 static audits. instructions: 263,437 static x86-64 instructions in the deployed binary (objdump of .text; ~33,500 of them from assembly source). Dynamic, measured: one ECDSA verification retires 360,030 user-space instructions (perf hardware counter, min of 5 rounds, deterministic to 5 digits – tools/instrcount.c). Kept as data: a former cover line, retired because a number that needs this much explaining belongs in an appendix. incidents: ~55 numbered production/consensus incidents false-accept class: ≥ 9 distinct defects + 2 structural findings audits: 2 independent, 11 + 11 findings; remediation incidents: 3 (own leak, disabled RPC, inert v2) capstone: MuHash3072 identical to Bitcoin Core at height 963,967 (165,726,554 txouts, 2,007,466,988,462,591 sat) worst night: 2026-09-01 – host frozen by awk, 2,596 resurrected coins (5,589.97458543 BTC) caught by muhash parity, repaired offline, re-identical at 965,085 perf: replay 7.8 -> 34.1 blk/s (4.39x) on identical spans; ecdsa_verify 115->~39us; libsecp256k1 gap 5.5x -> ~1.2x; sustained 10.0 blk/s at 537k-575k, full verification issuance check: the capstone amount – 2,007,466,988,462,591 sat at height 963,967 – is ~20,074,670 BTC against ~20,074,897 theoretical issuance: a ~227 BTC delta that is exactly where claimed-but-unclaimed and burned rewards should sit. Two independently maintained ledgers, the same satoshi count, and a shortfall that lands where Bitcoin's own economics says it must. end-to-end vs Core: NEVER MEASURED (published as an open item, twice) state at press time: live on mainnet, deploy 20260902a1, height ~965,124, "reload exact," muhash checked continuously

Appendix B: HOW TO READ A LOG THAT WAS WRITTEN AS A VACCINE

The method this report keeps praising is a format, and the format is copyable. Every incident in LOG.md follows one skeleton – teach it to a newcomer in five minutes and they can audit any claim in this report:

SYMPTOM first, in the machine's own words (the log line, verbatim). Never a paraphrase – the verbatim line is what future greps find.

TIMELINE, monotonic, with deploy letters so every event names the binary that produced it.

DAMAGE, split into damaged / not-damaged, with proofs for the second half ("0 overlaps," "CHAIN VERIFIED"). Optimistic recovery claims the not-damaged column like a bill.

ROOT CAUSE, at the layer where the fix goes – which is often not the layer where it was noticed (trigger / mechanism / mask in the 09-01 incident).

WRONG DIAGNOSES PRESERVED. The header-sync report's build-by-build table ("p" trimmed only the zero tail; "q" SEGVED on a contract the C side had wrong) reads like a lab notebook because the project banned the retrospective clean-up of confusion.

FIXES, each with the *test that fails against the unfixed code* – the negative control, the format's immune-system clause.

LESSONS, boxed, written as rules a future session can obey mechanically. A rule with no scar is a rule that gets argued away; the citation of the incident number is what makes it stick.

The corollary for readers: a defect log without the wrong-diagnoses section is marketing, and a fix list without negative controls is a wish list.

Appendix C: READING ORDER – AUDITING THIS REPORT AGAINST THE SOURCE

The complete source is public: <https://github.com/BobClawblaw/bitcoinmachinecode> Every number and quotation in this report lives in that tree, dated. Suggested route for the skeptic (about one focused afternoon):

1. `<https://github.com/BobClawblaw/bitcoinmachinecode>` – every artifact cited in this report is in that repository, dated. Start with:
2. `docs/devlog/PLAN.md` – the goal, in the project's own words, plus the "substantially REACHED" status block quoted in Part IX.
3. `docs/devlog/ASSESSMENT.md` – the honest-verdict document Parts II, IX, and X all quote; its "what would change this assessment" list is the project grading its own homework.
4. `docs/FEATURE_GAPS.md` – the ledger of what's missing and why; read the 2026-08-27 "this document was itself audited" note first to understand why its omissions are trustworthy.
5. Pick three incidents at random from `docs/devlog/LOG.md` (suggestions: #10 witness-stripped archive, #33 keep-up, #45 counter drift) and check each against `git log --oneline` around its date – commit messages carry the same root-cause prose, independently written.
6. `docs/audits/` – read the two audit reports before their responses, and check every "RESOLVED" against the code paths each response names.
7. Provenance, if the "zero human lines" claim is the one you care about: `git log --diff-filter=AM --format='%an' -- '*.asm'` attributes every assembly-touching commit to the AI-authoring identity (the two named committer identities in the history are both the operator's own, and 599 commit trailers name the AI co-author). The stronger evidence is structural: ~34k hand-written assembly instructions with zero vendored code – no third-party source tree to have copied from, and every constant table generated from an oracle (Appendix B). It is not a forensics-grade proof; it is the proof this report can actually print.
8. To reproduce the claims yourself: make test in `asm/`, the verification scripts under `validation/` against a scratch Core, and `daemon/utxo_setinfo` for the muhash on a real `datadir`. The instruments are ordinary tools; that is the point.

Epilogue: THE NODE IS STILL RUNNING

At 2026-09-02 12:11 UTC – as the final version of this report was being assembled – the binary `bitcoind.deploy-20260902a1` was live on mainnet (deploy letter `aq` by 17:42, the dead-weight fix landed), the `-Wall -Werror` campaign had closed with 393 warning sites fixed, six workers, gate 318/318 green. The archive keeps growing.

The evening proved something the report had only predicted. At 19:58 UTC the host went down – a Hermes agent job reached 50.6 GB on a 60 GB box, the kernel OOM-killed it, and a hard reset followed. The live node spent 39 seconds in uninterruptible sleep reading ~19 GB of run files back into a cold page cache, then reloaded at height 965,216 and caught up in 0.91 s. The acceptance run – 1 h 51 m and 34 GB into its block download, sixteen parallel workers mid-write – hit the SIGKILL case on an archive still being written. The boot check found two torn frames at the write frontier, trimmed `index.dat`'s empty records, and resumed at 9.7 MB/s with no peers banned. No block data was lost. The runner gained a `RESUME=1` mode so an interruption from outside the test costs a restart instead of the whole 34 GB download, and it now refuses to run if the daemon is still alive, so it cannot paper over the daemon itself dying. It also stopped counting "reorg candidate REJECTED (no action taken)" as a failure – that line is the node correctly refusing a lagging peer's shorter chain, routine over a multi-day mainnet sync. Round 8 continues on the same `datadir` from 20:14; it is now a cold start plus one unclean restart rather than a pure uninterrupted cold start, so an unqualified acceptance claim still wants a clean rerun.

The differential verification closed out the day: 63,000 whole-input cases and 400,000 EvalScript cases matching Core exactly, now covering real signatures Core itself produces – ECDSA over legacy and BIP143 sighashes, Schnorr over BIP341 key-path and BIP342 script-path sighashes with annexes, code separators, and two-leaf trees. The 71 Core-signed spends stay in the gate. MuSig2 and PSBT flows are the next layer.

The experiment is over. The node is the experiment's fossil – and it is following the chain. At 20:20 UTC, 1,034 commits on main, 1,281 across all refs, the UTXO count agreeing with Core's to the satoshi because the continuous muhash check would scream otherwise. Bitcoin Core, one directory over, keeps answering vector queries for a student that has, in 21 days, learned to agree with it – 165.7 million rows at a time, one MuHash at a time. Twenty-one days for twenty-one million sats, zero human lines – and the next block arrives in about nine minutes, verified by code that, at 2:47 this morning, could not have been written by anyone alive in time.

Appendix D: WHAT WE LEARNED, IN ONE PAGE

1. A clean replay proves the accept direction and nothing else. False accepts live in the inputs no honest miner ever mined.
2. Ground truth must be outside your belief boundary: a reference you didn't write, published vectors, or the chain. Agreement between your code and your tests is the sound a bug makes when it is inherited.
3. Every golden rule is a scar; every incident report is a vaccine; a gate that cannot prove it ran is a mood.
4. A documented assumption with no failing test is a scheduled outage.
5. Derived files must be derived; atomic renames are not optional; a stop that hangs turns a mistake into an outage; a recovery that "succeeds" is a claim, not a proof.
6. Profile before optimizing; state the Amdahl ceiling before writing the code; report negative results (the 4x prototype you killed by arithmetic counts); "we have not measured Core" is a publishable sentence.
7. Config options that lie are worse than missing; refusals must be named; metrics that count the wrong population are blindfolds.
8. Fail-closed, fail-sticky, and make the halt expensive to ignore.
9. The human is for standards, questions, and stopping rules. The machine is for being told "no, here is the file and line that prove it" – and for writing the report nobody feels like writing.
10. Twenty-one days, one machine, two intelligences, a chain of 965,000 blocks, and a ledger both of them can now read in the same language. Bitcoin's rules turned out to be a shared mother tongue. We both speak it now. 1,034 commits on main, 1,281 across all refs – the git record is the receipt.

– END –